Deep Reinforcement Learning

Contents

List of Figures

List of Tables

Abstract

This report is the final report for the deep reinforcement learning module coursework (INM707). GitHub repository: https://github.com/alexxcollins/Collins_Adib_INM707_CW. This report consists of 4 parts: Basic part, advanced part, Individual Part, and Finally the extra part PPO.

1 Basic Part

1.1 Define the environment

We work with a simple, relatively easy to solve environment to understand how exactly the Q-Learning algorithm works. To do that, we decided to work with a relatively small grid problem. This grid is originally 6x6 with only one agent. The goal of the agent is to go to work everyday, taking the least time possible, and collecting a croissant (breakfast) and cogs (tools for work) on the way.

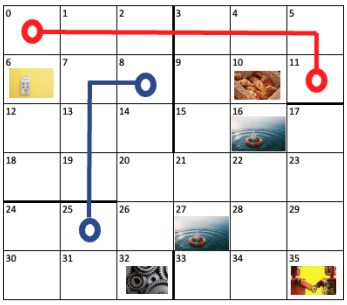


Figure 1: The grid game for the basic part

As we can see in figure 1, the starting position of our agent is at cell number 6, and the termination state is cell number 35. The grid has walls (black lines), which block the movement between cells in both directions. Moreover, the grid has tube trains which the agent can use as well. Finally the grid has 3 more types of cells, pond (bad), cogs(good) and croissants (good).

We should note that we make our grid flexible, in other words we can change the parameters and position of walls, tubes, start and end positions, rewards functions and dimensions of the grid... You can take a look at the **RobotEnv** class in **RobotEnvClass.py** file.

1.2 Transition function and reward function

1.2.1 Transition function

In our grid the state of the agent is represented by number of the cell in the grid. For instance the starting state in the default configuration is **6** and the termination state is **35**.

In reinforcement learning, moving from one state to another is called a transition. The transition structure is represented by: $s' = t(s, a)$, where s' is the state achieved from state s after taking an action a.

In each cell the agent can take an action $a \in A$, where **A** consists of 5 available actions: go up, down, right, left or staying in his position, as long as this doesn't mean leaving the grid or walking through a wall. Tubes also allow the agent to travel more than one suare.

However, we should note that the state is a number, but can also be represented as a coordinate (i.e.: x, y). For simplicity sake, we often use this convention in the code, and we have functions to transform cell to coordinate and vise-versa.

1.2.2 Reward function

Rewards are numerical values that agent receives when performing an action in the environment. The values can be positive or negative based on the action of the agent. So we can represent the function

by: $r' = r(s, a)$, where r' is the reward received after taking action **a** from state **s**.

From the table bellow, we can see that the value of each move to a normal state is a negative reward. The reason why we chose to give negative reward for these moves is to force the agent to find the shortest path. Falling into pond also has negative rewards. Moreover, the work and cogs cells have 20 rewards, while eating croissant before going to work has 30 rewards.

State	Available Actions
6	[6, 0, 7, 12]
0	[0, 1, 6, 23]
8	[8, 2, 7, 14]

Table 1: Example of some state and there possible actions.

Action	Reward
one timestep	-0.5
pond	-5
cogs	20
croissant	30
work (termination state)	20

Table 2: Rewards for taking actions

1.2.3 R Matrix

The available transition and reward of each move can be found in the **R Matrix**. We have 6x6 grid so the **R Matrix** consists of 36x36 rows and columns. (See Appendix A)

1.3 Set up parameters

We started by setting up our parameters which are: **alpha** (the learning rate), **gamma** (the discount factor), **epsilon** (trade off between exploitation and exploration) and **epsilon decay** which is a dictionary describing how epsilon decays (to increase exploitation) as the training progresses.

As initial value we chose to work with 0.9 for α, γ and ϵ. For the decay, if the uniform random value is greater than threshold in our case 0.5, ϵ is multiplied by 0.99999, otherwise it is multiplied by 0.9999. All the values are between 0 and 1.

1.4 Run the experiment

The first run of the experiment with initial parameters was poor. After 50,000 episodes the agent still didn't get to the final state (work) and total rewards were still negative. See Appendix A for more details.

1.5 Experiments with different parameters

After the miserable results from the initial parameters, we should start to work with different parameters.

After an initial search on alpha and gamma parameters individually (see notebook in codebase for more details). We narrowed down the range of best alpha to between 0.6 and 0.8, and gamma to between 0.7 and 0.9). We then performed a grid search to find the best combination of alpha and gamma. Results shown below.

Finally, we compared different values of epsilon. In training, average reward for low values of epsilon converged very quickly - this doens't necessarily show Q-convergence, but all values of epsilon allowed the model to converge at an optimum strategy.

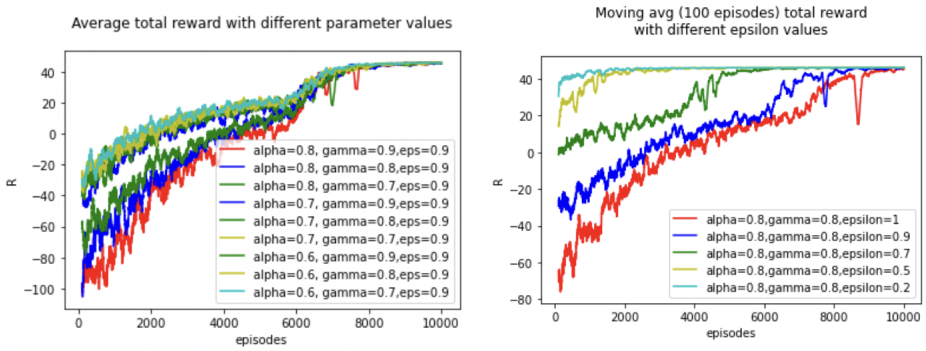


Figure 2: Hyper parameters tuning

After our grid search, we settled on the following parameters. Even though epsilon set at very low levels seemed to work well, we chose epsilon of 0.7 to allow some exploration initially.

Parameters	**Value**
α	0.8
γ	0.7
ϵ	0.7
ϵ-decay	0.9999, 0.9999, 0.5

Table 3: Rewards for taking actions

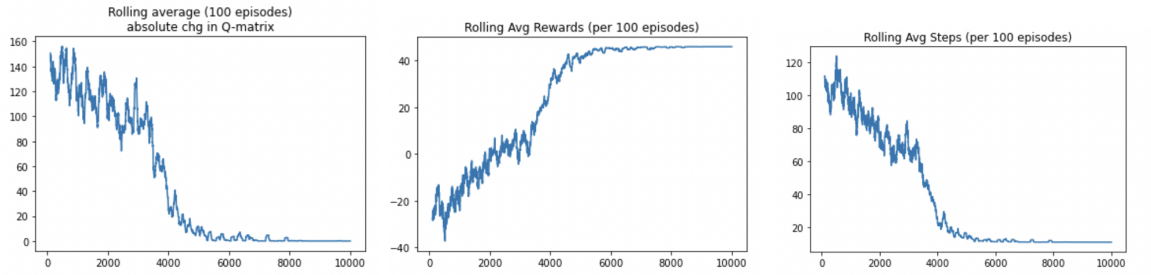


Figure 3: Final results

We can notice here that our agent learns to go to work, collect rewards in cell 10 only. We will discuss this result in next part.

1.6 Analyze the results quantitatively and qualitatively

First of all, results show that different values of hyperparameter $(\alpha, \gamma, \epsilon)$ have significant impact on the agent.

We saw with our first trial $(\alpha = 0.9, \gamma = 0.9, \epsilon = 0.9)$, the results were bad and the agent got stuck a loop and didn't achieve its terminal state. The values were to the maximum, and as we know when we want to study the impact of each of the hyperparamters on the learning process we can set the values to the boundaries and see.

To begin with, **alpha** or the learning rate is the factor that define the speed of the learning let's say. Sometimes it is better to learn faster and sometimes the learning must take time. Ideally, the learning rate can simply be defined as how much the agent accept the new value vs the old value. If α=0, there is no learning, the Q Values will never updated, if α=1 the agent will forget the previous value, and assume that the last value you got is the only one that matters.

Gamma or the discount factor is used to balance immediate and future reward. The discount factor factor determines how much the agent cares about rewards in the distant future relative to those in the immediate future. If γ=0 the agent will be completely myopic and only learn about actions that produce an immediate reward, if γ=1 the agent will evaluate each of its actions based on the sum total of all of its future rewards

Finally, **Epsilon** ϵ is the value the control the trade off between exploitation and exploration. As we know, the Q-Learning is said to be off-policy temporal difference control algorithm. To better understand the impact of **epsilon**, we have to understand the concept of exploration and exploitation first. In reinforcement learning, the agent try to discover the environment, and use the trial and error to learn. During these trails the agent have to select action. When the agent explores, it can improve its current knowledge and gain better rewards in the long run. However, when it exploits, it gets more reward immediately, even if it is a sub-optimal behavior. As the agent can't do both at the same time, there is a trade-off. If the value of **epsilon** is high (max=1) and the decay is slow, then the agent will take his time in exploring the environment and try to collect knowledge lets say. On the other hand, if the epsilon is low, and the decay is fast, the agent most of the time will take an action that returns the max reward.

From the initial values the results were too bad, so we decrease all the value together and after some experiments we got some mindful results.

We noticed that the agent managed to collect one of the rewards only. The rewards of **croissant** is 30, while the **cogs** is 20. When we created the game or the grid, our goal was to make the robot collect all the rewards and then go to work. For instance, go to 32 then 10, and 35, or 10, 32, 35. However, reinforcement learning and Q learning algorithm are based in **Markov Decision process**, which means the future is independent of the past given the present. Let's take the case where our agent use the path (32, 10, 35), in this case the agent have to go from 32 to 10, then use the same path to return to 35. So in this case when the agent is in cell number 21, he won't know where to go, 15 or 22, because our agent doesn't have any kind of memory. The same thing for the second option (10, 32, 35). The solution in this case is simple, instead of storing the number or the coordinates as state, we should add 2 more Boolean variables, one to keep track if we visited the **cogs** or not, another on for **croissant**. We should not here, to prevent the agent from visiting the same cell where we have rewards, after the agent collect the reward in particular cell, we set the value of visiting it again to **timestamp**.

1.7 Compare different algorithms

In order to better understand the **Q Learning** algorithm, we decided to work with different algorithms to solve this problem. These problem are: SARSA and Sophisticated Q Learning.

From the picture bellow, we can clearly see that **Q Learning** beats the other 2 algorithm. SARSA and Sophisticated Q learning aren't stable tho.

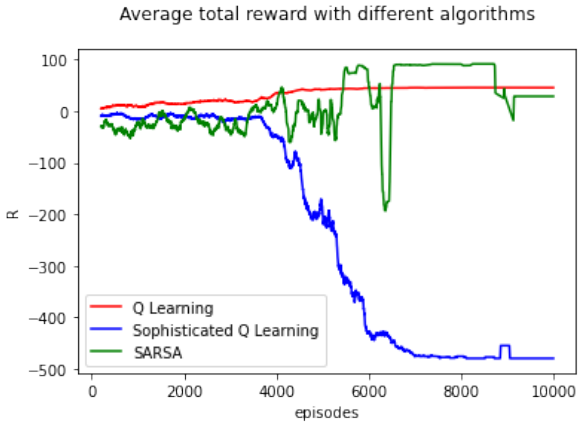


Figure 4: Comparing different reinforcement learning algorithms.

When the concept of reinforcement learning first raised, the **Temporal difference learning** was the first introduced. Usually the rewards are not immediately observed, TD learning is an unsupervised technique to predict a variable's expected value in a sequence of states. Two prominent algorithms are often used to expand on this topic and showcase the basics of reinforcement learning. Those algorithms are Q-learning and SARSA.

Q Learning uses **greedy policy** or **off-policy** to update the Q values. It takes the max from future actions.

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)\Big].$$

Figure 5: Q Learning Formula

On the other hand, **SARSA** which stands for **state-action-reward-state-action**, it's **on-policy** or **e-greedy-policy** which means it takes e-greedy action instead of taking the max.

Finally, **Sophisticated Q Learning** plays non-determinedly, in other words, it takes the next Q value based on probability. For instance if the next rewards for the next 3 possible states are: 80, 10, 10, the agent will choose the cell with the highest rewards with 80% chance.

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha\Big[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\Big].$$

Figure 6: SARSA Formula

2 Advanced Part

2.1 DQN on Snake: PyGame

Alex (mis-)spent many hours playing snake on a borrowed mobile phone at his school library. We think it is a good environment for Deep Q-Learning. Our game is on a 32 x 24 grid, surrounded by "walls". The snake is 3 blocks long and its aim is to eat a randomly placed **"rat"**. If it crashes into it's own body or a wall, it dies. If it eats the rat it scores a point and grows one block longer.

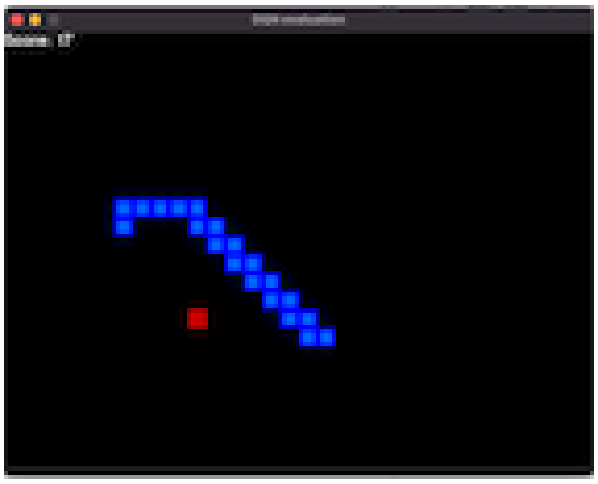


Figure 7: Snake searching for the rate

The environment alone is very big for classic Q-learning. The variable length snake, and randomly placed rat, leads to an explosion of possible environments. It is amenable to deep Q-learning however.

The agent observes pixel locations of objects. To ease compute, we chose to simplify the observation space significantly. We experimented with three different representations:

- in all cases agent perceived if rat was ahead, behind, left or right of the snake's head - "simple" case: agent sees which direction the snake is going in (UP, DOWN, LEFT, RIGHT) and if there is a collision possibility immediately next to the snake's head. - "relative" case: agent sees if there is a collision possibility immediately next to the snake's head. It also perceives if there is part of the snake's body anywhere ahead, behind, left or right of the snake's head. - "surrounding" case: snake perceives 5x5 grid of blocks around it's head. It distinguishes between empty space and an obstacle: either a wall or it's own body.

The size of observations is between 11 and 29 boolean values. Watching the snake play, inspired us to try different observation inputs, which is akin to us giving the snake a "soft" model in a model-free learning process.

Our neural net is very simple: one layer of 256 hidden neurons and no batch-normalisation. This can obviously be improved, but we wanted to focus on differences between reinforcement algorithm approaches.

2.2 Improvements

We experimented with Double-DQN and Dueling DQN algorithms. Dueling should be a more efficient architecture than DQN: splitting the network into a state-value and action value leads to more frequent updates of the state-value, and also more stability in action choice of a nearly greedy policy (Wang et al, 2015).

The shape of the agents observations is invariant to that of the environment size. This leads to an obvious question: can we use a curriculum approach to train the agent on successively bigger grids, and will this improve on starting with the full-sized grid? Our hypothesis was that during the initial phase of random action-choice, a smaller grid gives rise to more frequent collisions and rat-meals and allows the agent to learn these important features faster.

The curriculum consisted of training the agent on a 6x6 grid for 150 iterations. We increased epsilon decay to ensure that epsilon decayed to 0.1 regardless of whether the training was over 150 episodes, or 1500 episodes in DQN and DDQN methods above. We then transferred parameters to an agent training for 200 episodes on a 10x10 grid. Given that the agent had already learnt something we initiated epsilon as a smaller number: 0.8 instead of 0.9 seemed to work well. After the initial training, we tried different approaches. The first was zero-shot evaluation on the whole grid, the second was training on the full grid for 250 episodes. The third was training on a third smaller grid - this time 20 x 16 grid - for 250 episodes before moving on to a zero shot evaluation of the full grid. We only used the "surroundings" observation method for curriculum learning. This was partly to save time, but also because "surroundings" observation would be close to full knowledge of the environment on the smallest grid, and so we thought that the agent could learn even more effectively in the early stages.

We show results of DQN, Dueling-DQN, and Curriculum learning, across the different types of observations: simple (11 values), relative(12 values) and "surrounding" (29 values). For evaluation, we ran 30 episodes with agent choosing best action. We also set random seeds such that each game would be the same across all agents. The tables below show average rewards earned over all evaluation games.

stopping criteria	best parameters		1500 episodes		3rd round zero shot (curriculum)		4th round (curriculum)	
algorithm	DQN	Dueling_DQN	DQN	Dueling_DQN	DQN	Dueling DQN	DQN	Dueling DQN
simple	17.8	16.3	17.3	21.9	nan	nan	nan	nan
surroundings	19.2	19.5	20.3	12.4	8.1	22.9	14.2	27.3
relative	16.2	23.7	15.8	19.9	nan	nan	nan	nan

Figure 8: Average rewards over evaluatoin cycles

The dueling DQN did improve over DDQN, but not dramatically, and over our 1500 episode time horizon, dueling DQN actually got worse. The performance advantage of dueling DQN over DQN increases with the size of the action space. Our agent's action space is size three: straight, left or right. This is the most likely reason for relatively poor performance of dueling-DQN. See the annex for full training history of these algorithms. You can see how some improve and then deteriorate. Also the double + dueling DQN never performs well in (e-greedy) training, but does much better at evaluation.

On the other hand, the curriculum learning approach worked really well. We trained the DQN and DDQN agents on 1500 episodes of the full-sized grid. In curriculum learning, we trained 150 iterations on a 6x6 grid, followed by 200 iterations on a 10x10 grid. After that, evaluating the agent on the full-sized grid outperformed the other approaches, even thought the agent had never played on such a big grid before. There are also significant advantages of speed of training: the curriculum approach needed fewer episodes, and episodes were quicker given the smaller grid and shorter game length per episode.

3 Rllib

For rllib implementation, I chose pong and DQN. I trained on Pong Deterministic - which means that the every 4th frame is sampled: this is consistent with the original Deep Mind approach (Minh et al, 2013). I wanted to use images as input to make it a bit more challenging.

Pong is played against a computer player. The human (agent) controls a bat which has to return a ball and get it past the opponents bat. Games a played to 21. Rewards for the end of a game are calculated as agent score - opponent score, giving a maximum possible score of 21.

I chose DQN for practical reasons: it runs well on one GPU and limited cores (I even got good results on mac laptop) - PPO was harder to set up on the cloud compute I was using. I ran DQN with dueling and double.

Double DQN runs separate networks for action selection and Q-evaluation. The one target value in Deep Q-Learning is replace by the two values below. (Recall that the loss is the expectation of MSE between target and Q-value for current state - $L_i(\theta_i) = E_{s,a\sim\rho(.)}[(y_i - Q(s,a;\theta_i))^2]$).

$$Y_t^{\text{Q}} = R_{t+1} + \gamma Q(S_{t+1}, \underset{a}{\text{argmax}}\, Q(S_{t+1}, a; \boldsymbol{\theta}_t); \boldsymbol{\theta}_t)$$
$$Y_t^{\text{DoubleQ}} \equiv R_{t+1} + \gamma Q(S_{t+1}, \underset{a}{\text{argmax}}\, Q(S_{t+1}, a; \boldsymbol{\theta}_t); \boldsymbol{\theta}'_t)$$

Figure 9: Double DQN targets

Double Q-learning should avoid overestimation bias, where an agent gets lucky and things the action choice was the cause.

Given space restraints, I won't present dueling DQN in detail, but the network is split into two streams - one calculating a scalar to evaluate how good a state is and the other to evaluate action choice. This leads to more efficient training as we do not only update specific state-action pairs at each step. Dueling DQN also provides bigger benefits when action space is larger - this won't be relevant in pong where actions are in one dimension.

Implementing rllib on cloud infrastructure (I used JarvisLabs.io) was very challenging by itself. I encountered numerous problems, even pip install bringing in a version of the source code inconsistent with the github version (I had to use command sed bash command to modify the source code every time I set up a cloud session!).

Another problem I had was implementing ray.tune API. On Jarvis, it looked like it was running and then froze. I got it working on Colab, but the compute was limited and so I could not get meaningful results. On Colab, I ran grid search for pairs $\text{double}_dqn = [True, False], dueling_dqn[True, False], lr = [0.0001, 0.0002] but I didn't have enough resource to trust the answers and I do not present results of this exploration here.$

I did some pre-processing of the images to improve processing time: converting to greyscale and reducing the dimensions from 84x84 pixels to 42x42. As a starting point, I used suggested config settings from the rllib project here https://github.com/ray-project/ray/blob/master/rllib/tuned_examples/dqn/pong-dqn.yaml

When running the training, every 10 episodes, I saved (as a pickle file) a dictionary of lists containing the config object, max and average rewards per episode, checkpoint and epsilon information. This allows me to recreate training and results from these pkl files. One can recreate my results using the pkl files in github repo.

Without ray.tune, hyper parameter testing is very time consuming. I spent a huge amount of time (and money!) on different environments and parameters and present three here. First is with rllib suggested pong config parameters (see annex), second is with full sized images and third with learning rate of 0.0002 instead of 0.0001. Average reward per episode is shown in blue, max reward per episode is shown in orange.

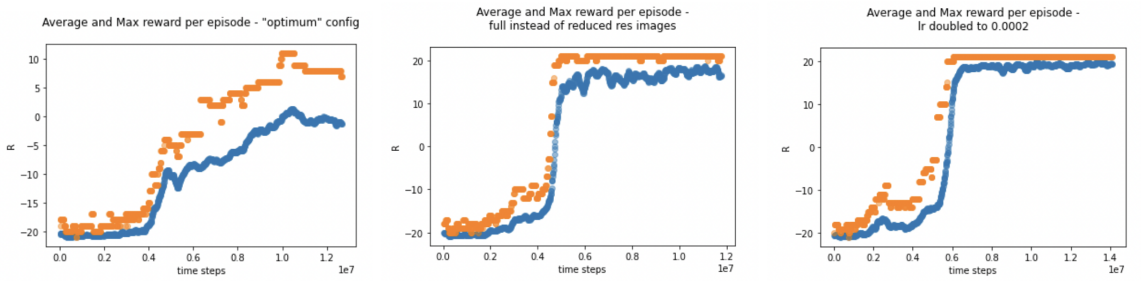


Figure 10: Training results

One can see that the original "optimum" config parameters were not sufficient to obtain a perfect score or even beat the opponent more than 50% of the time. I decided to try two experiments: increasing the preprocessed image size from 42x42 to 84x84 pixels (retaining greyscale) and doubling the learning rate from 0.0001 to 0.0002. Both were effective. To help parse the charts above, I present a comparison of the alternative configs, showing the first time step where a perfect score of 21 was achieved and the first timestep where the average score over the episode was above 20, hinting at close to perfect play.

	average score > 20	max score > 20
full resolution	nan	4971904
lr=0.0002	13392064	6054496

Figure 11: Quantying results

One can see that whilst, full resolution variant was the first to score a perfect score, after about 5mn time steps, it never performed as well, on average as the higher learning rate variant, which achieved near perfect play after 6mn time steps.

4 PPO Implementation

My implementation of PPO was heavily influence by the following repo: https://github.com/nikhilbarhate99/PPO-PyTorch. My method was simpler, implementing just a discrete action space: I will not be able to solve more complex continuous environments as a result. I kept the structure of the code similar to previous imlementations earlier in this CW, with a class for experience buffer (RolloutBuffer), a class called ActorCritic which defines the PyTorch nets used for the action choosing and evaluation, and finally a PPO class which implements PPO. Having seen the beautiful presentation of running training progress and parameters in the repo above, I just copied that to display very pretty results. I wrote custom visualisations.

PPO is one of the family of policy optimisation algorithms. The basic insight is that a policy gradient is measures and gradient descent (or ascent) is done on the policy. The standard loss function is below,

$$L^{PG}(\theta) = \hat{\mathbb{E}}_t \left[\log \pi_\theta(a_t \mid s_t) \hat{A}_t \right].$$

Figure 12: policy loss

The insight of PPO is to use a "clipped surrogate objective" and to then to perform multiple epochs of gradient ascent to perform each policy update.

The clipped loss is below. The clipping stops policy improvements from being too radical (which may lead to overall policy getting worse after update), but allows "corrective" policy adjustments to be unlimited. The calculations are much simpler than previous attempts at dealing with this problem (like in TRPO). This stability of policy updates allows multiple policy updates to be run after each collection of training data, which lowers cost of data collection.

PPO also uses several actors to collect data in parrallel.

In my implementation, I kept the clipping parameter the same as in the original paper (Schulman et all, 2017): 0.2.

$$L^{PG}(\theta) = \hat{\mathbb{E}}_t \left[\log \pi_\theta(a_t \mid s_t) \hat{A}_t \right].$$

Figure 13: clipped surrogate loss

I ran PPO for Cartpol-v1 (from OpenAI gym) and it converged to close to max reward of 400 after 50,000 time steps.

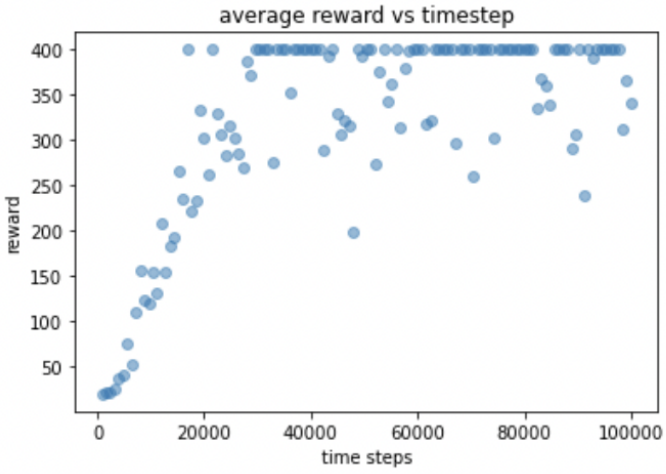


Figure 14: summary of cartpole training

5 References

Schulman, J., Wolski, F., Dhariwal, P., Radford, A. and Klimov, O., 2017. Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347.

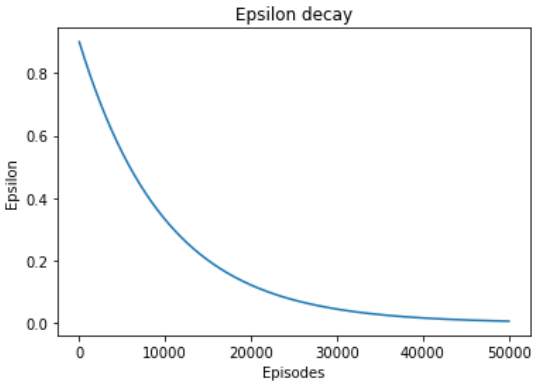
Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D. and Riedmiller, M., 2013. Playing atari with deep reinforcement learning. arXiv preprint arXiv:1312.5602.

Github, PPO-PyTorch, viewed 20 April 2022, ¡https://github.com/nikhilbarhate99/PPO-PyTorch¿.

Appendix A Basic Part

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
0	-0.5	-0.5	NaN	NaN	NaN	NaN	-0.5	NaN	NaN	NaN	NaN	-0.5	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
1	-0.5	-0.5	-0.5	NaN	NaN	NaN	NaN	-0.5	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2	NaN	-0.5	-0.5	NaN	NaN	NaN	NaN	NaN	-0.5	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
3	NaN	NaN	NaN	-0.5	-0.5	NaN	NaN	NaN	NaN	-0.5	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
4	NaN	NaN	NaN	-0.5	-0.5	-0.5	NaN	NaN	NaN	NaN	30.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
5	NaN	NaN	NaN	NaN	-0.5	-0.5	NaN	NaN	NaN	NaN	NaN	-0.5	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
6	-0.5	NaN	NaN	NaN	NaN	NaN	-0.5	-0.5	NaN	NaN	NaN	NaN	-0.5	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
7	NaN	-0.5	NaN	NaN	NaN	NaN	-0.5	-0.5	-0.5	NaN	NaN	NaN	NaN	-0.5	NaN	NaN	NaN	NaN	NaN	NaN	NaN
8	NaN	NaN	-0.5	NaN	NaN	NaN	NaN	-0.5	-0.5	NaN	NaN	NaN	NaN	NaN	-0.5	NaN	NaN	NaN	NaN	NaN	NaN
9	NaN	NaN	NaN	-0.5	NaN	NaN	NaN	NaN	NaN	-0.5	30.0	NaN	NaN	NaN	NaN	-0.5	NaN	NaN	NaN	NaN	NaN
10	NaN	NaN	NaN	NaN	-0.5	NaN	NaN	NaN	NaN	-0.5	-0.5	-0.5	NaN	NaN	NaN	NaN	-5.0	NaN	NaN	NaN	NaN
11	NaN	NaN	NaN	NaN	NaN	-0.5	NaN	NaN	NaN	NaN	30.0	-0.5	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
12	NaN	NaN	NaN	NaN	NaN	NaN	-0.5	NaN	NaN	NaN	NaN	NaN	-0.5	-0.5	NaN	NaN	NaN	NaN	-0.5	NaN	NaN
13	NaN	NaN	NaN	NaN	NaN	NaN	NaN	-0.5	NaN	NaN	NaN	NaN	-0.5	-0.5	-0.5	NaN	NaN	NaN	NaN	-0.5	NaN
14	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	-0.5	NaN	NaN	NaN	NaN	-0.5	-0.5	NaN	NaN	NaN	NaN	NaN	-0.5
15	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	-0.5	NaN	NaN	NaN	NaN	NaN	-0.5	-5.0	NaN	NaN	NaN	NaN
16	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	30.0	NaN	NaN	NaN	NaN	-0.5	-5.0	-0.5	NaN	NaN	NaN
17	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	-5.0	-0.5	NaN	NaN	NaN
18	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	-0.5	NaN	NaN	NaN	NaN	NaN	-0.5	-0.5	NaN
19	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	-0.5	NaN	NaN	NaN	NaN	-0.5	-0.5	-0.5
20	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	-0.5	NaN	NaN	NaN	NaN	-0.5	-0.5

	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35
21	-0.5	-0.5	NaN	NaN	NaN	NaN	-5.0	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
22	-0.5	-0.5	-0.5	NaN	NaN	NaN	NaN	-0.5	NaN	NaN	NaN	NaN	NaN	NaN	NaN
23	NaN	-0.5	-0.5	NaN	NaN	NaN	NaN	NaN	-0.5	NaN	NaN	NaN	NaN	NaN	NaN
24	NaN	NaN	NaN	-0.5	-0.5	NaN	NaN	NaN	NaN	-0.5	NaN	NaN	NaN	NaN	NaN
25	NaN	NaN	NaN	-0.5	-0.5	-0.5	NaN	NaN	NaN	NaN	-0.5	NaN	NaN	NaN	NaN
26	NaN	NaN	NaN	NaN	-0.5	-0.5	-5.0	NaN	NaN	NaN	NaN	20.0	NaN	NaN	NaN
27	-0.5	NaN	NaN	NaN	NaN	-0.5	-5.0	-0.5	NaN	NaN	NaN	NaN	-0.5	NaN	NaN
28	NaN	-0.5	NaN	NaN	NaN	NaN	-5.0	-0.5	-0.5	NaN	NaN	NaN	NaN	-0.5	NaN
29	NaN	NaN	-0.5	NaN	NaN	NaN	NaN	-0.5	-0.5	NaN	NaN	NaN	NaN	NaN	20.0
30	NaN	NaN	NaN	-0.5	NaN	NaN	NaN	NaN	NaN	-0.5	-0.5	NaN	NaN	NaN	NaN
31	NaN	NaN	NaN	NaN	-0.5	NaN	NaN	NaN	NaN	-0.5	-0.5	20.0	NaN	NaN	NaN
32	NaN	NaN	NaN	NaN	NaN	-0.5	NaN	NaN	NaN	NaN	-0.5	-0.5	NaN	NaN	NaN
33	NaN	NaN	NaN	NaN	NaN	NaN	-5.0	NaN	NaN	NaN	NaN	NaN	-0.5	-0.5	NaN
34	NaN	NaN	NaN	NaN	NaN	NaN	NaN	-0.5	NaN	NaN	NaN	NaN	-0.5	-0.5	20.0
35	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	-0.5	NaN	NaN	NaN	NaN	-0.5	20.0



With the previous parameters, the results were terrible.

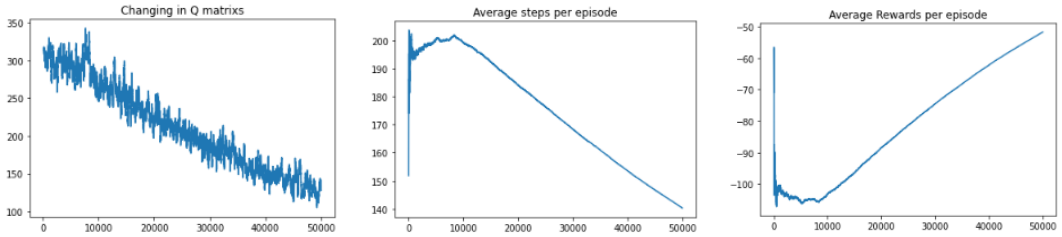


Figure 15: Results of first experiment

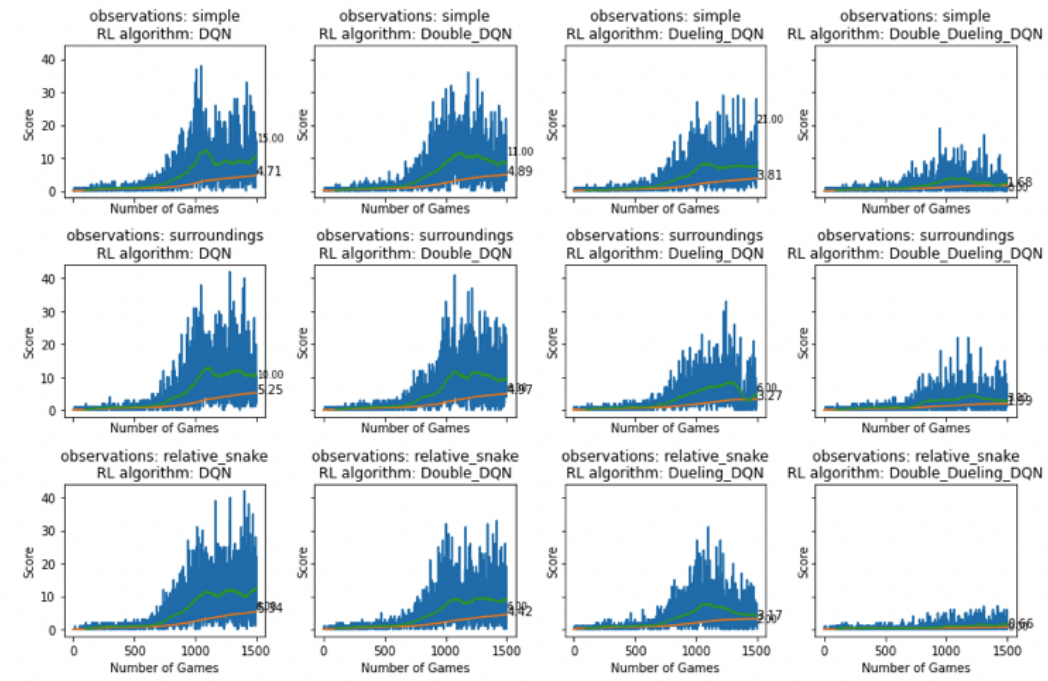
```
path = find_path(Q_hist[-1])
r, steps = calculate_rewards(q_learning.R, path)
print("Path: ",path)
print("Path in grid coords: ", path_in_grids(path, q_learning.dims))
print("Total Rewards: ", r)
print("Number of steps: ",steps)
```

```
The Agent stucked into a loop
Path:  [6, 7, 8, 2]
Path in grid coords:  [(1, 0), (1, 1), (1, 2), (0, 2)]
Total Rewards:  -1.5
Number of steps:  3
```

Figure 16: Changing in Q matrix

As can be seen above, after 50,000 episodes the agent still can't get to work, or collect resources with a higher value then his time cost.

Appendix B Advanced Part



stopping criteria	1500 episodes				best parameters				350 episodes			
algorithm	DQN	Double_DQN	Dueling_DQN	Double_Dueling_DQN	DQN	Double_DQN	Dueling_DQN	Double_Dueling_DQN	DQN	Double_DQN	Dueling_DQN	Double_Dueling_DQN
simple	17.3	10.4	21.9	12.8	17.8	15.4	16.3	21.3	13.9	2.8	22.8	7.6
surroundings	20.3	10.4	12.4	14.5	19.2	16.3	19.5	23.5	1.6	15.3	3.4	0.2
relative	15.8	12.8	19.9	0.8	16.2	15.2	23.7	4.0	5.4	20.9	15.5	17.7

	0
3rd_round_dueling_zero_shot.pth	22.9
3rd_round_dueling.pth	17.1
4th_round_dueling_zero_shot.pth	19.1
4th_round_dueling.pth	27.3
3rd_round_dqn_zero_shot.pth	8.1
3rd_round_dqn.pth	7.4
4th_round_dqn_zero_shot.pth	14.6
4th_round_dqn.pth	14.2

Appendix C Basic Part Code

RobotEnvClass.py

```
import numpy as np
import pandas as pd
from abc import ABC, abstractmethod
import random


# This is a class for creating the Robot environment

class RobotEnv(ABC):
```

```
    # Constructor for initialization, takes as input the parameters of the
env such as start, end, rewards, positions...
    def __init__(self,
                 dims=(6, 6),
                 rewards={'r_time': -1, 'r_pond': -15,
                          'r_croissant': 200, 'r_cogs': 200,
                          'r_work': 15},
                 start=(1, 0),
                 end=(5, 5),
                 positions={'pond': [(2, 4), (4, 3)],
                            'cogs': [(5, 2)],
                            'croissant': [(1, 4)]},
                 tubes=[[(0, 0), (3, 5)], [(1, 2), (4, 1)]],
                 walls=[[(0, 2), (0, 3)], [(1, 2), (1, 3)],
                        [(2, 2), (2, 3)], [(1, 5), (2, 5)],
                        [(3, 0), (4, 0)],
                        [(3, 1), (4, 1)], [(5, 2), (5, 3)]],
                 epsilon_decays={'epsilon_threshold': 0.5,
                                 'epsilon_decay1': 0.99999,
                                 'epsilon_decay2': 0.9999},
                 max_steps=1000,
                 max_episodes=1000,
                 random_seed=42
                 ):
        self._dims = dims  # first number is height, second is width
        self._rewards = rewards
        self._start = start
        self._end = end
        self._tubes = tubes
        self._walls = walls
        self._epsilon_decays = epsilon_decays
        self._positions = positions
        self._max_steps = max_steps
        self._max_episodes = max_episodes

        # initialize the grid, R_matrix and Q_matrix
        self._initialize_grid()
        self._initialize_R_matrix()
        self._initialize_Q_matrix()

        self.rng = np.random.default_rng(random_seed)

    # getters and setters

    # getter and setter for dims
    @property
    def dims(self):
        return self._dims

    @dims.setter
    def dims(self, dims):
        # When changing the dims we have to re-initialize: grid, R, Q
        self._dims = dims
        self._initialize_grid()
        self._initialize_R_matrix()
        self._initialize_Q_matrix()
```

```
    # getter and setter for start
    @property
    def start(self):
        return self._start

    @start.setter
    def start(self, start):
        self._start = start

    # getter and setter for end
    @property
    def end(self):
        return self._end

    @end.setter
    def end(self, end):
        self._end = end

    # getter and setter for tubes
    @property
    def tubes(self):
        return self._tubes

    @tubes.setter
    def tubes(self, tubes):
        self._tubes = tubes
        self._initialize_grid()
        self._initialize_R_matrix()
        self._initialize_Q_matrix()
        self.__initializeTunnels()

    # getter and setter for max steps
    @property
    def max_steps(self):
        return self._max_steps

    @max_steps.setter
    def max_steps(self, max_steps):
        self._max_steps = max_steps

    # getter and setter for max episodes
    @property
    def max_episodes(self):
        return self._max_episodes

    @max_episodes.setter
    def max_episodes(self, max_episodes):
        self._max_episodes = max_episodes

    # getter and setter for max walls
    @property
    def walls(self):
        return self._walls

    @walls.setter
```

```
    def walls(self, walls):
        self._walls = walls

    # getter and setter for epsilon_decays
    @property
    def epsilon_decays(self):
        return self._epsilon_decays

    @epsilon_decays.setter
    def epsilon_decays(self, epsilon_decays):
        self._epsilon_decays = epsilon_decays

    # some property with only getter, in other words the user can't modify those propert
    @property
    def grid(self):
        return self._grid

    @property
    def R(self):
        return self._R

    @property
    def Q(self):
        return self._Q

    @property
    def rewards(self):
        return self._rewards

    @walls.setter
    def rewards(self, rewards):
        self._rewards = rewards
        self._initialize_grid()
        self._initialize_R_matrix()
        self._initialize_Q_matrix()

    @property
    def positions(self):
        return self._positions

    # function to initialize the grid, the propose of the grid is for visualization
    def _initialize_grid(self):
        self._grid = np.zeros(self.dims)
        for position in self._positions:
            for pos in self._positions[position]:
                self._grid[pos[0], pos[1]] = self._rewards['r_' + position]
                # print(position, ":", pos, ":", self._grid[pos[0], pos[1]])

        self._grid[self._end[0], self._end[1]] = self._rewards['r_work']
        # print(self._grid)

    # function to print a grid visualisation for the task
    def visualise_world(self):
        W = self._dims[1] * 2 + 1
        H = self._dims[0] * 2 + 1
        # create empty grid
```

```
        rows = [['    '] * W for r in range(H)]

        # add dots for spaces where agent can move
        for i in range(W):
            for j in range(H):
                if i % 2 == 1 and j % 2 == 1:
                    rows[j][i] = '.   '

        # border round the grid
        rows[0] = ['X   ' for c in rows[0]]
        rows[H - 1] = ['X   ' for c in rows[H - 1]]
        for i in range(1, H - 1):
            rows[i][0] = 'X   '
            rows[i][W - 1] = 'X   '

        # insert walls
        for w in self._walls:
            rows[(w[0][0] + w[1][0]) + 1][w[0][1] + w[1][1] + 1] = 'X   '

        # insert tubes
        for n, t in enumerate(self._tubes):
            rows[t[0][0] * 2 + 1][t[0][1] * 2 + 1] = 'T{}  '.format(n)
            rows[t[1][0] * 2 + 1][t[1][1] * 2 + 1] = 'T{}  '.format(n)

        # insert start, end
        for label, p in [('S   ', self._start), ('E   ', self._end)]:
            rows[p[0] * 2 + 1][p[1] * 2 + 1] = label

        # insert features
        symbol = {'pond': 'P   ', 'cogs': 'G   ', 'croissant': 'C   '}
        for key in self._positions:
            for p in self._positions[key]:
                rows[p[0] * 2 + 1][p[1] * 2 + 1] = symbol[key]

        return self._create_viz_string(rows)

    # function to join strings in rows and add key
    def _create_viz_string(self, rows):
        s = '\n' + ''.join([''.join(r) + '\n' for r in rows])
        s += '\n'
        s += 'key:\n'
        s += 'S  = start location for agent\n'
        s += 'E  = end location for agent\n'
        s += '.  = empty cell\n'
        s += 'X  = boundary or wall beetween cells\n'
        s += 'Tn = nth Tube start or end. Agent can travel between the two Tn in one tim
        s += 'P  = pond: falling in is cold and wet\n'
        s += 'G  = cog: agent is rewarded for collecting\n'
        s += 'C  = croissant: agent is rewarded for collecting\n'

        return s

    # initialize the rewards matrix
    def _initialize_R_matrix(self):
        d1 = self.dims[0]
        d2 = self.dims[1]
```

```
        self._R = np.empty((d1 * d2, d1 * d2))
        self._R.fill(np.nan)  # Fastest way to initilize R matrix

        # call some methods instead to write all the function here, cleaner and better f
        self.__fillPossibleActions()
        self.__initializeTunnels()
        self.__initializeCogs()
        self.__initializePonds()
        self.__initializeCroissants()
        self.__initializeGoalPoint()
        self.__initializeWalls()

    # helper function, used in initialization methods
    def move_to(self, l, feature):
        """ creates a list of tuples with cell agent is moving to and cell agent is movi
        cell = feature[0] * self._dims[1] + feature[1]

        if feature[0] > 0:  # cell not on top edge
            l.append((cell - self._dims[1], cell))
        if feature[0] < self._dims[0] - 1:  # cell not on bottom edge
            l.append((cell + self._dims[1], cell))
        if feature[1] > 0:  # cell not left hand edge
            l.append((cell - 1, cell))
        if feature[1] < self._dims[1] - 1:  # cell not on right hand edge
            l.append((cell + 1, cell))

        return l

    # function to fill all the possible moves
    def __fillPossibleActions(self):
        # All moves where reward is .self_rewards['r_time'] for action.
        ones = []
        for i in range(self._dims[0]):  # iterating over rows
            for j in range(self._dims[1]):  # iterating across columns
                cell = i * self._dims[1] + j
                if j != self._dims[1] - 1:
                    # move right unless agent is on right edge
                    ones.append((cell + 1, cell))
                if i != self._dims[0] - 1:
                    # move up if not in top row
                    ones.append((cell + self._dims[1], cell))
                if i != 0:
                    # move down not in bottom row
                    ones.append((cell - self._dims[1], cell))
                if j != 0:
                    # move left if not on left edge
                    ones.append((cell - 1, cell))
                # staying still is possible, why not?
                ones.append((cell, cell))

        ones = tuple(zip(*ones))
        self._R[ones] = self._rewards['r_time']
        # for i in range(self._dims[0]):
        #     for j in range(self._dims[1]):
        #         cell = i * self._dims[0] + j
        #         self._R[(cell, cell)] = self.rewards['r_time']
```

```
            #         #self._R[(cell, cell)] = np.nan
            #         #self._R[(cell, cell)] = 0

        # initialize the goal rewards
        def __initializeGoalPoint(self):
            end_cell = self._end[0] * self._dims[1] + self._end[1]
            ends = self.move_to([], self._end)
            ends.append([end_cell, end_cell])
            ends = tuple(zip(*ends))
            self._R[ends] = self._rewards['r_work']

        # initialize the Tunnels
        def __initializeTunnels(self):
            tubes_cells = []
            for tubes in self._tubes:
                tubes_cell = []
                for tube in tubes:
                    cell_nb = tube[0] * self._dims[1] + tube[1]
                    tubes_cell.append(cell_nb)
                # print(tubes_cell)
                tubes_cells.append(tuple(tubes_cell))
            for cell in tubes_cells.copy():
                # print(cell)
                tubes_cells.append((cell[0], cell[1]))

            tubes_cells = tuple(zip(*tubes_cells))
            self._R[tubes_cells] = self._rewards['r_time']

        # initialize the Cogs rewards
        def __initializeCogs(self):
            cogs = []
            for cog in self._positions['cogs']:
                cogs = self.move_to(cogs, cog)

            cogs = tuple(zip(*cogs))
            self._R[cogs] = self._rewards['r_cogs']

        # initialize the Ponds rewards
        def __initializePonds(self):
            # don't fall in the pond!
            # print(self._positions['pond'])
            ponds = []
            for pond in self._positions['pond']:
                p = pond[0] * self._dims[1] + pond[1]
                ponds = self.move_to(ponds, pond)
                ponds.extend([(p, p)])

            ponds = tuple(zip(*ponds))
            self._R[ponds] = self._rewards['r_pond']

        # initialize the Croissant rewards
        def __initializeCroissants(self):
            croissants = []
            for croissant in self._positions['croissant']:
                croissants = self.move_to(croissants, croissant)
```

```
        croissants = tuple(zip(*croissants))
        self._R[croissants] = self._rewards['r_croissant']

    # finally, construct the walls
    def __initializeWalls(self):
        for wall in self._walls:
            cell0 = wall[0][0] * self._dims[1] + wall[0][1]
            cell1 = wall[1][0] * self._dims[1] + wall[1][1]
            wall_in_matrix = ((cell0, cell1), (cell1, cell0))
            self._R[wall_in_matrix] = np.nan

    # display the matrix as pandas dataframe
    def display_matrix(self, matrix, start=None, end=None):
        pd.set_option("display.max_columns", None)
        display(pd.DataFrame(matrix).loc[start:end, start:end])

    # initialize the Q matrix with the same shape as R matrix
    def _initialize_Q_matrix(self):
        self._Q = np.zeros(self._R.shape)

    # function to run over only one episode, takes as input alpha, gamma and epsilon
    @abstractmethod
    def run_episode(self, Q, alpha, gamma, epsilon):
        pass

    @abstractmethod
    def learn(self, alpha, gamma, epsilon):
        pass

    # method to get adjacent cells
    def _get_adjacent_cells(self, cell):
        cells = np.where([~np.isnan(self.R[cell,])])[1]
        return cells

    def _get_actions(self, R, Q, s):
        """
        Returns best and all available actions as lists
        """
        available = np.where(~np.isnan(R[s]))[0]
        q_vals = [Q[s, a] for a in available]
        best = available[np.where(q_vals == np.max(q_vals))[0]]
        # change the type from np array to list
        available = available.tolist()
        best = best.tolist()
        return available, best

    def _get_greedy_action(self, epsilon, available, best):
        """
        Given epsilon, and available and best actions,
        Pick an appropriate action.
        """
        if self.rng.uniform() > epsilon:
            a = self.rng.choice(best)
        else:
            a = self.rng.choice(available)
        return a
```

Q_Learning.py

```
class Q_Learning(RobotEnv):

    def __init__(self,
                 dims=(6, 6),
                 rewards={'r_time': -1, 'r_pond': -15, 'r_croissant': 200, 'r_cogs': 200
                 start=(1, 0),
                 end=(5, 5),
                 positions={'pond': [(2, 4), (4, 3)], 'cogs': [(5, 2)], 'croissant': [(1
                 tubes=[[(0, 0), (3, 5)], [(1, 2), (4, 1)]],
                 walls=[[(0, 2), (0, 3)], [(1, 2), (1, 3)], [(2, 2), (2, 3)], [(1, 5), (
                        [(3, 1), (4, 1)], [(5, 2), (5, 3)]],
                 epsilon_decays={'epsilon_threshold': 0.5, 'epsilon_decay1': 0.99999, 'e
                 max_steps=1000,
                 max_episodes=1000,
                 random_seed=42
                 ):
        super().__init__(dims,
                         rewards,
                         start,
                         end,
                         positions,
                         tubes,
                         walls,
                         epsilon_decays,
                         max_steps,
                         max_episodes,
                         random_seed
                         )

    def run_episode(self, Q, alpha, gamma, epsilon):

        R_tot = 0
        # print(self._start)
        s = self._start[0] * self._dims[1] + self._start[1]
        # action_hist = np.array([s])
        action_hist = [s]
        # action_hist = 0
        goal_state = self._end[0] * self._dims[1] + self._end[1]
        # Q = self._Q
        R = self._R.copy()
        # print("Starting Point: ", s)
        # print("End Point: ", goal_state)

        # some lists to keep track of visited cogs and croissant cells
        # to prevent the agent from re-visit them in the same episode to collect resourc
        cogs_visited = []
        croissant_visited = []

        cogs_cells = [cog_position[0] * self._dims[1] + cog_position[1] for cog_position

        croissant_cells = [croissant_position[0] * self._dims[1] + croissant_position[1]
                           self.positions['croissant']]
```

```
        for i in range(self._max_steps):
            # actions selection
            available, best = self._get_actions(R, Q, s)
            a = self._get_greedy_action(epsilon, available, best)
            action_hist.append(a)
            # action_hist += 1
            s_old = s
            s = a

            # update Q:
            Q[s_old, a] = Q[s_old, a] + alpha * (R[s_old, a] +
                                                 gamma * Q[s, :].max() -
                                                 Q[s_old, a])

            # update total accumulated reward for this episode
            R_tot += R[s_old, a]
            if a in cogs_cells or a in croissant_cells:
                cells = self._get_adjacent_cells(a)
                for cell in cells:
                    R[cell, a] = self._rewards['r_time']

            if s == goal_state:
                break

        return Q, R_tot, action_hist

    # function to run Q learning algorithm
    # off-policy
    # greedy policy
    # we want to record the path agent followed on each episode which we do with a jagge

    def learn(self, alpha, gamma, epsilon):
        Q = self._Q.copy()
        Rtot = np.empty(shape=self.max_episodes)
        a_hist = np.empty(shape=(self.max_episodes), dtype=np.object_)
        Q_hist = np.empty(shape=(self.max_episodes, self._Q.shape[0], self._Q.shape[1]))

        for episode in range(self.max_episodes):
            Q, r, action_hist = self.run_episode(Q, alpha, gamma, epsilon)
            # Rtot.append(r)
            Rtot[episode] = r
            a_hist[episode] = action_hist
            Q_hist[episode, :, :] = Q

            if epsilon > self.epsilon_decays['epsilon_threshold']:
                epsilon *= self.epsilon_decays['epsilon_decay1']
            else:
                epsilon *= self.epsilon_decays['epsilon_decay2']

        return a_hist, Q_hist, Rtot
```

SARSA.py

```
import numpy as np
import pandas as pd
from abc import ABC, abstractmethod
import random
```

```
from RobotEnvClass import RobotEnv

class SARSA_learning(RobotEnv):

    def __init__(self,
                 dims=(6, 6),
                 rewards={'r_time': -1, 'r_pond': -15, 'r_croissant': 200, 'r_cogs': 200
                 start=(1, 0),
                 end=(5, 5),
                 positions={'pond': [(2, 4), (4, 3)], 'cogs': [(5, 2)], 'croissant': [(1
                 tubes=[[(0, 0), (3, 5)], [(1, 2), (4, 1)]],
                 walls=[[(0, 2), (0, 3)], [(1, 2), (1, 3)], [(2, 2), (2, 3)], [(1, 5), (
                        [(3, 1), (4, 1)], [(5, 2), (5, 3)]],
                 epsilon_decays={'epsilon_threshold': 0.5, 'epsilon_decay1': 0.99999, 'e
                 max_steps=1000,
                 max_episodes=1000,
                 random_seed=42
                 ):
        super().__init__(dims,
                         rewards,
                         start,
                         end,
                         positions,
                         tubes,
                         walls,
                         epsilon_decays,
                         max_steps,
                         max_episodes,
                         random_seed
                         )

    def run_episode(self, Q, alpha, gamma, epsilon):
        R_tot = 0
        # print(self._start)
        s = self._start[0] * self._dims[1] + self._start[1]
        goal_state = self._end[0] * self._dims[1] + self._end[1]
        R = self._R.copy()
        action_hist = [s]
        # some lists to keep track of visited cogs and croissant cells
        # to prevent the agent from re-visit them in the same episode to collect resourc
        cogs_visited = []
        croissant_visited = []

        cogs_cells = [cog_position[0] * self._dims[1] + cog_position[1] for cog_position

        croissant_cells = [croissant_position[0] * self._dims[1] + croissant_position[1]
                           self.positions['croissant']]

        for i in range(self._max_steps):
            # actions selection
            available, best = self._get_actions(R, Q, s)
            a = self._get_greedy_action(epsilon, available, best)
            action_hist.append(a)

            s_old = s
```

```
            s = a
            _available, _best = self._get_actions(R, Q, s)
            _a = self._get_greedy_action(epsilon, _available, _best)
            Q[s_old, a] = Q[s_old, a] + alpha * (R[s_old, a] +
                                                 gamma * Q[s, _a] -
                                                 Q[s_old, a])

            # update total accumulated reward for this episode
            R_tot += R[s_old, a]
            # update total accumulated reward for this episode
            R_tot += R[s_old, a]
            if a in cogs_cells or a in croissant_cells:
                cells = self._get_adjacent_cells(a)
                for cell in cells:
                    R[cell, a] = self._rewards['r_time']
            if s == goal_state:
                break

        return Q, R_tot, action_hist

    '''
    function to run SARSA learning algorithm
    SARSA stands for State-Action-Reward-State-Action
    on-policy
    e-greedy policy
    '''

    def learn(self, alpha, gamma, epsilon):
        Q = self._Q.copy()
        Rtot = np.empty(shape=self.max_episodes)
        a_hist = np.empty(shape=(self.max_episodes), dtype=np.object_)
        Q_hist = np.empty(shape=(self.max_episodes, self._Q.shape[0], self._Q.shape[1]))
        for episode in range(self.max_episodes):
            Q, r, action_hist = self.run_episode(Q, alpha, gamma, epsilon)
            Rtot[episode] = r
            a_hist[episode] = action_hist
            Q_hist[episode, :, :] = Q

            if epsilon > self.epsilon_decays['epsilon_threshold']:
                epsilon *= self.epsilon_decays['epsilon_decay1']
            else:
                epsilon *= self.epsilon_decays['epsilon_decay2']

        return a_hist, Q_hist, Rtot
```

Sophisticated_Q_learning.py

```
import numpy as np
import pandas as pd
from abc import ABC, abstractmethod
import random

from RobotEnvClass import RobotEnv


class Q_Learning_Randomness(RobotEnv):
```

```
    def __init__(self,
                 dims=(6, 6),
                 rewards={'r_time': -1, 'r_pond': -15, 'r_croissant': 200, 'r_cogs': 200
                 start=(1, 0),
                 end=(5, 5),
                 positions={'pond': [(2, 4), (4, 3)], 'cogs': [(5, 2)], 'croissant': [(1
                 tubes=[[(0, 0), (3, 5)], [(1, 2), (4, 1)]],
                 walls=[[(0, 2), (0, 3)], [(1, 2), (1, 3)], [(2, 2), (2, 3)], [(1, 5), (
                        [(3, 1), (4, 1)], [(5, 2), (5, 3)]],
                 epsilon_decays={'epsilon_threshold': 0.5, 'epsilon_decay1': 0.99999, 'e
                 max_steps=1000,
                 max_episodes=1000,
                 random_seed=42
                 ):
        super().__init__(dims,
                         rewards,
                         start,
                         end,
                         positions,
                         tubes,
                         walls,
                         epsilon_decays,
                         max_steps,
                         max_episodes,
                         random_seed
                         )

    def run_episode(self, Q, alpha, gamma, epsilon):

        R_tot = 0
        # print(self._start)
        s = self._start[0] * self._dims[1] + self._start[1]
        goal_state = self._end[0] * self._dims[1] + self._end[1]
        # Q = self._Q
        R = self._R.copy()
        action_hist = [s]
        # print("Starting Point: ", s)
        # print("End Point: ", goal_state)

        cogs_cells = [cog_position[0] * self._dims[1] + cog_position[1] for cog_position

        croissant_cells = [croissant_position[0] * self._dims[1] + croissant_position[1]
                           self.positions['croissant']]

        for i in range(self._max_steps):
            # actions selection
            available, best = self._get_actions(R, Q, s)
            a = self._get_greedy_action(epsilon, available, best)
            action_hist.append(a)

            s_old = s
            s = a

            # First get the available actions from rhe current state
            _available, _best = self._get_actions(R, Q, s)
            # calculate the probability distribution accroding to the Q-values
```

```
            sum_list = sum(_available)
            propa = [round((value / sum_list) * 100) for value in _available]
            # Choise the next state non-deterministically
            non_deterministic_action = random.choices(_available, weights=propa)
            # update Q:
            Q[s_old, a] = Q[s_old, a] + alpha * (R[s_old, a] +
                                                 gamma * non_deterministic_action[0] -
                                                 Q[s_old, a])

            # update total accumulated reward for this episode
            R_tot += R[s_old, a]
            if a in cogs_cells or a in croissant_cells:
                cells = self._get_adjacent_cells(a)
                for cell in cells:
                    R[cell, a] = self._rewards['r_time']

            if s == goal_state:
                break

        return Q, R_tot, action_hist

    # function to run Q learning algorithm
    # off-policy
    # greedy policy
    def learn(self, alpha, gamma, epsilon):
        Q = self._Q.copy()
        Rtot = np.empty(shape=self.max_episodes)
        a_hist = np.empty(shape=(self.max_episodes), dtype=np.object_)
        Q_hist = np.empty(shape=(self.max_episodes, self._Q.shape[0], self._Q.shape[1]))

        for episode in range(self.max_episodes):
            Q, r, action_hist = self.run_episode(Q, alpha, gamma, epsilon)
            Rtot[episode] = r
            a_hist[episode] = action_hist
            Q_hist[episode, :, :] = Q

            if epsilon > self.epsilon_decays['epsilon_threshold']:
                epsilon *= self.epsilon_decays['epsilon_decay1']
            else:
                epsilon *= self.epsilon_decays['epsilon_decay2']

        return a_hist, Q_hist, Rtot
```

RL_utils.py

```
import numpy as np
import matplotlib.pyplot as plt


def find_path(Q):
    state = 6
    path = [state]
    # path = []
    end_state = False

    while not end_state:
        old_state = state
```

```
        state = np.where(Q[old_state,] == Q[old_state,].max())[0][0]
        if state not in path:
            path.append(state)
            if state == 35:
                end_state = True

        elif state == old_state:
            print("The Agent chose to stay in his position")
            end_state = True

        else:
            print("The Agent stucked into a loop")
            end_state = True

    return path

def calculate_rewards(R_matrix, path):
    r = 0
    steps = 0
    for idx in range(len(path)-1):
        r += R_matrix[path[idx], path[idx+1]]
        steps += 1
    return r,steps

def path_in_grids(path, dims):
    return [(cell//dims[1], cell%dims[0]) for cell in path]

def moving_average(a, n=3) :
    ret = np.cumsum(a, dtype=float)
    ret[n:] = ret[n:] - ret[:-n]
    return ret[n - 1:] / n

def average_R_display(Rtot, rolling_periods=None):
    """display average of total rewards over episodes

    rolling_periods: None or int
        if None, display the cumulative average.
        if int, display the moving average over n episodes
            this is more helpful for seeing convergence
            as earlier very large/small R-values are discarded.
    """
    if rolling_periods:
        Rtot_avg = moving_average(Rtot, n=rolling_periods)
        x = np.arange(rolling_periods - 1, len(Rtot))
    else:
        Rtot_avg = (Rtot.cumsum() / np.arange(1, len(Rtot) + 1))[20:]
        x = np.arange(20, len(Rtot))

    plt.plot(x, Rtot_avg)
    plt.title("Average Rewards per episode")
    plt.show()

def steps_plot(a_hist, rolling_periods=None):
    """
    rolling_periods: None or int
        if None, display the cumulative average.
```

```
        if int, display the moving average over n episodes
            this is more helpful for seeing convergence
            as earlier very large/small R-values are discarded.
    """
    y = [len(h) for h in a_hist]
    x = np.arange(len(a_hist))

    if rolling_periods:
        y_avg = moving_average(y, n=rolling_periods)
        x = np.arange(rolling_periods - 1, len(y))
    else:
        y_avg = (np.cumsum(y) / np.arange(1, len(y) + 1))[20:]
        x = np.arange(20, len(y))
    plt.plot(x, y_avg)
    plt.title("Average steps per episode")
    plt.show()


def plot_Q_changing(Q_hist, absolute=True, rolling_periods=None):

    if absolute:
        difference_Qs = np.absolute(Q_hist[1:,:,:] - Q_hist[:-1, :, :])
    else:
        difference_Qs = Q_hist[1:,:,:] - Q_hist[:-1, :, :]

    Q_plot = np.sum(difference_Qs, axis=(1,2))
    if rolling_periods:
        x = np.arange(rolling_periods - 1, len(Q_plot))
        Q_plot = moving_average(Q_plot, n=rolling_periods)
    else:
        x = np.arange(len(Q_plot))

    plt.plot(x, Q_plot)
    plt.title("Changing in Q matrixs")
    plt.show()
```

Appendix D Advanced Part Code

Game.py

```
import pygame
import random
from enum import Enum
from collections import namedtuple
import numpy as np
import sys

pygame.init()
font = pygame.font.Font(None, 25)


class Direction(Enum):
    RIGHT = 1
    LEFT = 2
    UP = 3
```

```
    DOWN = 4


Point = namedtuple('Point', 'x, y')

# rgb colors
WHITE = (255, 255, 255)
RED = (200, 0, 0)
BLUE1 = (0, 0, 255)
BLUE2 = (0, 100, 255)
BLACK = (0, 0, 0)


class SnakeGameAI:

    def __init__(self, width=640, height=480, block_size=20,
                 UI=False,
                 game_speed=100, window_title="RL Snake",
                 rat_reset_seeds=np.random.randint(0,100000,size=1000)
                ):
        """
        AI Snake Game Environment
        :param width (int): width of the game window
        :param height (int): height of the game window
        :param block_size (int): block size of each block on the window
        :param UI (bool): set to true to display pyGame UI. Can set to
            false if training in cloud environment with no video UI
        :param game_speed (int): the speed of the game
        :param window_title (str): the title of the window
        :param seed (int): random seed
        """
        self._width = width
        self._height = height
        self._block_size = block_size
        self._game_speed = game_speed
        self._rat_reset_seeds = iter(rat_reset_seeds)
        self.UI = UI
        # init display
        if self.UI:
            self.display = pygame.display.init()
            self.display = pygame.display.set_mode((self._width, self._height))
            pygame.display.set_caption(window_title)
            self.clock = pygame.time.Clock()

        self.direction = None
        self.snake_head = None
        self.snake_body = None
        self.rat = None
        self.score = 0
        self.frame_iteration = 0
        self.reset()

    # Getter and setter
    @property
    def width(self):
        return self._width
```

```
@width.setter
def width(self, width):
    if width > 0:
        self._width = width

@property
def height(self):
    return self._height

@height.setter
def height(self, height):
    if height > 0:
        self._height = height

@property
def block_size(self):
    return self._block_size

@block_size.setter
def block_size(self, block_size):
    if block_size > 0:
        self._block_size = block_size

@property
def game_speed(self):
    return self._game_speed

@game_speed.setter
def game_speed(self, game_speed):
    if game_speed > 0:
        self._game_speed = game_speed

@property
def rat_reset_seeds(self):
    return self._rat_reset_seeds

@rat_reset_seeds.setter
def rat_reset_seeds(self, seed_array):
     self._rat_reset_seeds = iter(seed_array)

# method to reset the environment
def reset(self):
    # init game state
    self.direction = Direction.RIGHT

    self.snake_head = Point(self._width / 2, self._height / 2)
    self.snake_body = [self.snake_head,
                       Point(self.snake_head.x
                             - self._block_size, self.snake_head.y),
                       Point(self.snake_head.x
                             - (2 * self._block_size), self.snake_head.y)]

    self.score = 0
    self.rat = None
    self._place_rat()
```

```
        self.frame_iteration = 0

    # method to place a rat randomly on the screen
    def _place_rat(self):
        random.seed(next(self._rat_reset_seeds))
        x = random.randint(0, (self._width - self._block_size)
                           // self._block_size) * self._block_size
        y = random.randint(0, (self._height - self._block_size)
                           // self._block_size) * self._block_size
        self.rat = Point(x, y)
        if self.rat in self.snake_body:
            self._place_rat()

    # quit the game
    def end_game(self):
        pygame.quit()

    # method to place a step based on action
    def play_step(self, action):
        self.frame_iteration += 1
        # 1. collect user input
        if self.UI:
            for event in pygame.event.get():
                if event.type == pygame.QUIT:
                    self.end_game()

        # 2. move
        self._move(action)  # update the head
        self.snake_body.insert(0, self.snake_head)

        # 3. check if game over
        reward = 0
        game_over = False
        if (self.is_collision()
            # what does this do??
            or self.frame_iteration > 100 * len(self.snake_body)):
            game_over = True
            reward = -10
            return reward, game_over, self.score

        # 4. place new food or just move
        if self.snake_head == self.rat:
            self.score += 1
            reward = 10
            self._place_rat()
        else:
            self.snake_body.pop()

        # 5. update ui and clock
        if self.UI:
            self._update_ui()
            self.clock.tick(self._game_speed)
        # 6. return game over and score
        return reward, game_over, self.score

    # method to check if there is a collision or not
```

```
def is_collision(self, pt=None):
    if pt is None:
        pt = self.snake_head
    # hits boundary
    if (pt.x > self._width - self._block_size
        or pt.x < 0 or pt.y > self._height - self._block_size
        or pt.y < 0):
        return True
    # hits itself
    if pt in self.snake_body[1:]:
        return True

    return False

def relative_danger(self):
    """returns flattend array of shape (4,)
    Indices represent [ahead, right, left, behind]
    Values are one if collision is in that direction
    """
    # First create ndarray of shape (2,2) to represent absolute position
    # of collision relative to head
    # indices represent: [[up, right],
    #                     [left, down]]
    point_l = Point(self.snake_head.x - self.block_size, self.snake_head.y)
    point_r = Point(self.snake_head.x + self.block_size, self.snake_head.y)
    point_u = Point(self.snake_head.x, self.snake_head.y - self.block_size)
    point_d = Point(self.snake_head.x, self.snake_head.y + self.block_size)

    A = np.zeros((2,2))

    if self.is_collision(point_u):
        A[0,0] = 1
    if self.is_collision(point_d):
        A[1,1] = 1
    if self.is_collision(point_l):
        A[1,0] = 1
    if self.is_collision(point_r):
        A[0,1] = 1

    # TODO: code below is repeated in relative_body() should make into its
    # own method to prevent repitition.

    # R is position of snakes body RELATIVE to direction snake is moving
    # To transform A to R, rotate A depeding on direction snake is moving
    if self.direction == Direction.UP:
        R = A
    if self.direction == Direction.LEFT:
        R = np.rot90(A, 1)
    if self.direction == Direction.DOWN:
        R = np.rot90(A, 2)
    if self.direction == Direction.RIGHT:
        R = np.rot90(A, 3)

    return R.flatten()
```

```
    def relative_body(self):
        """Returns ndarray of shape (4,)
        Indices represent [ahead, right, left, behind]
        Values are one if a segment of the snakes body is in that direction relative
        to the snake, zero if not.
        """
        # First create ndarray of shape (2,2) to represent absolute position
        # of snake body relative to head
        # indices represent: [[up, right],
        #                     [left, down]]
        A = np.zeros((2,2))
        for segment in self.snake_body[1:]:
            if segment == self.snake_head.x:
                if segment < self.snake_head.y:
                    A[0,0] = 1
                if segment > self.snake_head.y:
                    A[1,1] = 1
            if segment == self.snake_head.y:
                if segment < self.snake_head.x:
                    A[1,0] = 1
                if segment > self.snake_head.x:
                    A[0,1] = 1

        # R is position of snakes body RELATIVE to direction snake is moving
        # To transform A to R, rotate A depeding on direction snake is moving
        if self.direction == Direction.UP:
            R = A
        if self.direction == Direction.LEFT:
            R = np.rot90(A, 1)
        if self.direction == Direction.DOWN:
            R = np.rot90(A, 2)
        if self.direction == Direction.RIGHT:
            R = np.rot90(A, 3)

        return R.flatten()

    # method to update the user interface
    def _update_ui(self):

        # snake body parts are dark blue squares (BLUE1) with
        # light blue borders (BLUE2)
        # calculate adjustment for BLUE2
        border = self._block_size // 5
        # size of each side of inner square
        border2 = self._block_size - 2 * border

        self.display.fill(BLACK)

        for pt in self.snake_body:
            pygame.draw.rect(self.display,
                             BLUE1,
                             pygame.Rect(pt.x, pt.y,
                                         self._block_size,
                                         self._block_size))
            pygame.draw.rect(self.display,
                             BLUE2,
```

```
                                pygame.Rect(pt.x + border,
                                            pt.y + border,
                                            border2, border2))

        pygame.draw.rect(self.display,
                         RED,
                         pygame.Rect(self.rat.x, self.rat.y,
                                     self._block_size,
                                     self._block_size))

        text = font.render("Score: " + str(self.score), True, WHITE)
        self.display.blit(text, [0, 0])
        pygame.display.flip()

    # method to move based on action
    def _move(self, action):
        # [straight, right, left]

        clock_wise = [Direction.RIGHT, Direction.DOWN,
                      Direction.LEFT, Direction.UP]
        idx = clock_wise.index(self.direction)

        if np.array_equal(action, [1, 0, 0]):
            new_dir = clock_wise[idx]  # no change
        elif np.array_equal(action, [0, 1, 0]):
            next_idx = (idx + 1) % 4
            new_dir = clock_wise[next_idx]  # right turn r -> d -> l -> u
        else:  # [0, 0, 1]
            next_idx = (idx - 1) % 4
            new_dir = clock_wise[next_idx]  # left turn r -> u -> l -> d

        self.direction = new_dir

        x = self.snake_head.x
        y = self.snake_head.y
        if self.direction == Direction.RIGHT:
            x += self._block_size
        elif self.direction == Direction.LEFT:
            x -= self._block_size
        elif self.direction == Direction.DOWN:
            y += self._block_size
        elif self.direction == Direction.UP:
            y -= self._block_size

        self.snake_head = Point(x, y)

    def _create_env_array(self):
        """Change environment to an array where there are padded boundaries
        and snake body parts. Anything that kills snakes with collision is
        represented by a 1, anything that doesn't is represented as 0

        We pad with 3 rows/columns of walls. This is because a terminal state
        give snake location as part of the wall. To avoid errors, x or y +- 2
        has to be an index in the array.
        """
        ar_height = int(self.height / self.block_size)
```

```
        ar_width = int(self.width / self.block_size)

        self.env_array = np.ones((ar_width + 6, ar_height + 6))
        self.env_array[3:-3, 3:-3] = 0

        for point in self.snake_body:
            x, y = self._coord_to_ar_idx(point)
            self.env_array[x,y] = 1

    def _coord_to_ar_idx(self, point):
        """Convert x,y coord to array index.

        Devide by block size and add 2.

        E.g. if x = 0, snake is still in the game. So with grid padded by 2,
        x index has to be = 2.
        """
        return (int(point.x / self.block_size) + 3,
                int(point.y / self.block_size) + 3)

    def surroundings(self):
        """Return 5x5 array of immediate surroundings"""

        self._create_env_array()
        x, y = self._coord_to_ar_idx(self.snake_head)
        surroundings = self.env_array[x - 2: x +3,
                                      y - 2: y + 3]
        # get_observations method creates list and converts to array.
        # may as well supply it with a list from .surroundings()

        return surroundings
```

ExperienceReplayBuffer.py

```
import numpy as np
import random
from collections import namedtuple
from collections import deque
from abc import ABC, abstractmethod

Experience = namedtuple("Experience", ('state', 'action', 'reward', 'next_state', 'done'


class Buffer(ABC):

    def __init__(self, buffer_size, batch_size):
        self._buffer_size = buffer_size
        self._batch_size = batch_size
        self._buffer = []

    # getter and setter
    @property
    def buffer_size(self):
        return self._buffer_size

    @property
    def batch_size(self):
```

```
        return self._batch_size

    def __len__(self):
        return len(self._buffer)

    @abstractmethod
    def append(self, experience):
        pass

    @abstractmethod
    def sample(self):
        pass


class ExperienceReplayBuffer(Buffer):

    def __init__(self, buffer_size, batch_size):
        super(ExperienceReplayBuffer, self).__init__(buffer_size, batch_size)
        self._buffer = deque(maxlen=self._buffer_size)

    def append(self, experience):
        # if len(_buffer) = buffer_size, then .append kicks of first object in deque
        # Note from AC: clever use of deques, Khalil!
        self._buffer.append(experience)

    def sample(self):
        if len(self._buffer) > self._batch_size:
            buffer_sample = random.sample(self._buffer, self._batch_size)

        else:
            buffer_sample = random.sample(self._buffer, len(self._buffer))
        return buffer_sample
```

Model.py

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import os

class DQN(nn.Module):
    def __init__(self, input_size, hidden_size, output_size):
        super().__init__()
        self.linear1 = nn.Linear(input_size, hidden_size)
        self.linear2 = nn.Linear(hidden_size, output_size)

    def forward(self, x):
        x = F.relu(self.linear1(x))
        x = self.linear2(x)
        return x


class DuelingDQN(nn.Module):

    def __init__(self, input_size, hidden_size, output_size):
        super(DuelingDQN, self).__init__()
```

```
        self.feature_extractor_network = nn.Sequential(
            nn.Linear(input_size, hidden_size),
            nn.ReLU()
        )

        self.value_q_network = nn.Sequential(
            nn.Linear(hidden_size, hidden_size),
            nn.ReLU(),
            nn.Linear(hidden_size, 1),
        )

        self.advantage_q_network = nn.Sequential(
            nn.Linear(hidden_size, hidden_size),
            nn.ReLU(),
            nn.Linear(hidden_size, output_size),
        )

    def forward(self, X):
        features = self.feature_extractor_network(X)
        advantages = self.advantage_q_network(features)
        values = self.value_q_network(features)
        q_vals = values + (advantages - advantages.mean())
        return q_vals
```

Agent.py

```
import torch
import torch.nn as nn
import torch.optim as optim
import os
import random
import numpy as np
from abc import ABC, abstractmethod
from collections import deque

from Game import SnakeGameAI, Direction, Point
from Model import DQN, DuelingDQN
from ExperienceReplayBuffer import Experience, ExperienceReplayBuffer


class Agent:

    def __init__(self,
                 get_observation='relative_snake',
                 learning_rate=0.001,
                 gamma=0.9,
                 epsilon=0.9,
                 epsilon_decay=[0.999, 0.995],
                 memory_capacity=100000,
                 batch_size=1000,
                 update_frequency=20,
                 double_dqn=True,
                 dueling_dqn=False,
                 prioritized_memory=False,
                 greedy=True,
                 game = SnakeGameAI()):
        """
```

```
        Constractor for Agent class
        ---------------
        Parameters:
        ---------------
        get_observation (string): either 'relative_snake', 'surroundings', 'simple'
            There are three different representations of the snake's perception of
            its environment.
                'relative_snake' defines observed state as whether the snakes body is
                ahead, right, left or behind the head, and whether there is an obstacle
                next to the snakes head.
                'surroundings' defines a 5x5 square around the snakes head and whether
                it contains walls or snake's body.
                'simple' defines whether snake is next to an obstacle and which directio
                the snake is pointing.
                All functions return whether the rat is ahead/behind or left/right of
                the snake's head.
        learning_rate (float): learning rate of the deep network between 0 and 1
        gamma (float): gamma value between 0 and 1
        epsilon (float): epsilon value between 0 and 1
        epsilon_decay (list): consists of 2 values for decay, between 0 and 1
        memory_capacity (int64): the capacity of the buffer
        update_frequency (int): the frequency of updating the target network
        double_dqn (bool): True if we want to use Double Q Learning option
        prioritized_memory (bool): True if we want to use prioritized buffer
        greedy: set to False if you want the snake to make optimal policy choices
        game: allows the game to be constructed and shared between different agents -
              this is necessary to prevent kernal restarts
        """
        self.observation_mode = get_observation
        self.lr = learning_rate
        self.gamma = gamma
        self.epsilon = epsilon
        self.epsilon_decay = epsilon_decay
        self.batch_size = batch_size
        self.double_dqn = double_dqn
        self.dueling_dqn = dueling_dqn
        self.prioritized_memory = prioritized_memory
        self.update_frequency = update_frequency
        self.number_episodes = 0
        self.number_timesteps = 0
        self.greedy = greedy

        self.game = game

        # TODO - get rid of prioritized_memory: we don't use it
        if prioritized_memory:
            pass
        else:
            self.replay_memory = ExperienceReplayBuffer(memory_capacity, batch_size)

        if self.observation_mode == 'relative_snake':
            self.get_observation = self.get_observation_relative_snake
            input_nodes = 12
        if self.observation_mode == 'surroundings':
            self.get_observation = self.get_observation_surroundings
            input_nodes = 29
```

```
        if self.observation_mode == 'simple':
            self.get_observation = self.get_observation_simple
            input_nodes = 11
        # define two neural network, one for policy and one for target
        if self.dueling_dqn:
            self.policy_net = DuelingDQN(input_nodes, 256, 3)
            self.target_net = DuelingDQN(input_nodes, 256, 3)
        else:
            self.policy_net = DQN(input_nodes, 256, 3)
            self.target_net = DQN(input_nodes, 256, 3)

        self._synchronize_q_networks()
        self.target_net.eval()

        # Define the optimizer and the loss function
        self.optimizer = optim.Adam(self.policy_net.parameters(), lr=self.lr)
        self.criterion = nn.MSELoss()

    # get observation and return np array of size
    def get_observation_surroundings(self) -> np.array:
        """Return summary of agents observation as
        shape (29,) np.array.

        Values are integers representing boolean values.
        [25 element list representing danger or safety in surrounding squares,
        rat left of snake's head,
        rat right of snake's head,
        rat above snake's head,
        rat below snake's head]
        """
        head = self.game.snake_body[0]
        block_size = self.game.block_size

        rat_up = self.game.rat.x < self.game.snake_head.x
        rat_down = self.game.rat.x > self.game.snake_head.x
        rat_right = self.game.rat.y > self.game.snake_head.y
        rat_left = self.game.rat.y < self.game.snake_head.y

        surroundings = self.game.surroundings()

        # depending on direction the snake is facing, rotate surroundings
        # and relative_rate arrays.
        if self.game.direction == Direction.UP:
            relative_rat = [rat_up, rat_down, rat_left, rat_right]
        if self.game.direction == Direction.LEFT:
            surroundings = np.rot90(surroundings, 1)
            relative_rat = [rat_left, rat_right, rat_down, rat_up]
        if self.game.direction == Direction.DOWN:
            surroundings = np.rot90(surroundings, 2)
            relative_rat = [rat_down, rat_up, rat_right, rat_left]
        if self.game.direction == Direction.RIGHT:
            surroundings = np.rot90(surroundings, 3)
            relative_rat = [rat_right, rat_left, rat_up, rat_down]

        return np.concatenate((surroundings.flatten(), np.array(relative_rat, dtype=int)
```

```
    def get_observation_relative_snake(self) -> np.array:
        """Return summary of agents observation as
        shape (12,) np.array.

        Values are integers representing boolean values.
        [4 elemnts representing if the snakes body is above, below, right or left of snal
        4 elements representing if snake will crash into something,
        4 elements representing where the rat is relative to the snake]
        """
        rat_up = self.game.rat.x < self.game.snake_head.x
        rat_down = self.game.rat.x > self.game.snake_head.x
        rat_right = self.game.rat.y > self.game.snake_head.y
        rat_left = self.game.rat.y < self.game.snake_head.y

        relative_body = self.game.relative_body()

        relative_danger = self.game.relative_danger()

        # depending on direction the snake is facing, rotate relative_rat arrays.
        if self.game.direction == Direction.UP:
            relative_rat = [rat_up, rat_down, rat_left, rat_right]
        if self.game.direction == Direction.LEFT:
            relative_rat = [rat_left, rat_right, rat_down, rat_up]
        if self.game.direction == Direction.DOWN:
            relative_rat = [rat_down, rat_up, rat_right, rat_left]
        if self.game.direction == Direction.RIGHT:
            relative_rat = [rat_right, rat_left, rat_up, rat_down]

        return np.concatenate((relative_body,
                               relative_danger,
                               np.array(relative_rat, dtype=int)))

    # get observation and return np array of size 11
    def get_observation_simple(self) -> np.array:
        """Return summary of agents observation as
        shape (11,) np.array.

        Values are integers representing boolean values.
        [danger straight, danger right, danger left,
        snake moving left, right, up, down,
        rat left of snake's head,
        rat right of snake's head,
        rat above snake's head,
        rat below snake's head]
        """
        head = self.game.snake_body[0]
        block_size = self.game.block_size

        point_l = Point(head.x - block_size, head.y)
        point_r = Point(head.x + block_size, head.y)
        point_u = Point(head.x, head.y - block_size)
        point_d = Point(head.x, head.y + block_size)

        dir_l = self.game.direction == Direction.LEFT
        dir_r = self.game.direction == Direction.RIGHT
        dir_u = self.game.direction == Direction.UP
```

```
        dir_d = self.game.direction == Direction.DOWN

        state = [
            # Danger straight
            (dir_r and self.game.is_collision(point_r)) or
            (dir_l and self.game.is_collision(point_l)) or
            (dir_u and self.game.is_collision(point_u)) or
            (dir_d and self.game.is_collision(point_d)),

            # Danger right
            (dir_u and self.game.is_collision(point_r)) or
            (dir_d and self.game.is_collision(point_l)) or
            (dir_l and self.game.is_collision(point_u)) or
            (dir_r and self.game.is_collision(point_d)),

            # Danger left
            (dir_d and self.game.is_collision(point_r)) or
            (dir_u and self.game.is_collision(point_l)) or
            (dir_r and self.game.is_collision(point_u)) or
            (dir_l and self.game.is_collision(point_d)),

            # Move direction
            dir_l,
            dir_r,
            dir_u,
            dir_d,

            # Rat location
            self.game.rat.x < self.game.snake_head.x,  # rat left
            self.game.rat.x > self.game.snake_head.x,  # rat right
            self.game.rat.y < self.game.snake_head.y,  # rat up
            self.game.rat.y > self.game.snake_head.y  # rat down
        ]

        return np.array(state, dtype=int)

    # method to push to the memory
    def remember(self, state, action, reward, next_state, done):
        experience = Experience(state, action, reward, next_state, done)
        self.replay_memory.append(experience)
        # self.replay_memory.append((state, action, reward, next_state, done))
        # popleft if memory_capacity is reached

    # method to return random action
    def _random_action(self):
        action = [0, 0, 0]
        random_move = random.randint(0, 2)
        action[random_move] = 1
        return action

    # method to return a greedy action, takes as input state
    def _epsilon_greedy_action(self, state):
        action = [0, 0, 0]
        self.policy_net.eval()
        with torch.no_grad():
            # convert the state to a tensor:
```

```
            state = torch.tensor(state, dtype=torch.float)
            # get the prediction from the policy net:
            prediction = self.policy_net(state)

        self.policy_net.train()
        greedy_move = torch.argmax(prediction).item()
        action[greedy_move] = 1

        return action

    # method to update the value of epsilon
    def update_policy(self):
        if self.epsilon > 0.5:
            self.epsilon *= self.epsilon_decay[0]
        else:
            self.epsilon *= self.epsilon_decay[1]

        if self.epsilon < 0.05:
            self.epsilon = 0.05

    # function to return epsilon greedy policy
    def choose_action(self, state) -> list:
        # random moves: tradeoff exploration / exploitation
        if random.uniform(0, 1) < self.epsilon and self.greedy:
            action = self._random_action()
        else:
            action = self._epsilon_greedy_action(state)

        # Alex: I think we really should update epsilon after each
        # episode, otherwise it's hard to compare across different
        # algos etc cos epsilon will decay to different values over 100 episodes say,
        # and we can't tell how random the agents actions are
        # self.update_policy()
        return action

    def _synchronize_q_networks(self):
        self.target_net.load_state_dict(self.policy_net.state_dict())

    def _soft_update_target_q_network_parameters(self) -> None:
        """Soft-update of target q-network parameters
        with the local q-network parameters."""
        for target_param, local_param in zip(self.target_net.parameters(),
                                             self.policy_net.parameters()):
            target_param.data.copy_(self.lr * local_param.data
                                    + (1 - self.lr) * target_param.data)

    # method to save the model
    def save_model(self, file_name):
        model_folder_path = './model'
        if not os.path.exists(model_folder_path):
            os.makedirs(model_folder_path)

        file_name = os.path.join(model_folder_path, file_name)
        torch.save({'state_dict': self.policy_net.state_dict(),
                    'optimizer': self.optimizer.state_dict(),
                    }, file_name)
```

```
    # method to load the weight
    def load_model(self, file_name):
        model_path = os.path.join('./model', file_name)
        if os.path.isfile(model_path):
            print("=> loading checkpoint... ")
            # self.model.load_state_dict(torch.load(model_path))
            checkpoint = torch.load(model_path)
            self.policy_net.load_state_dict(checkpoint['state_dict'])
            self.optimizer.load_state_dict(checkpoint['optimizer'])
            self._synchronize_q_networks()
            print("done !")
        else:
            print("no checkpoint found...")

    # method to return a sample from the memory
    def get_memory_sample(self):
        if self.prioritized_memory:
            pass
        else:
            mini_sample = self.replay_memory.sample()
            states, actions, rewards, next_states, dones = zip(*mini_sample)

            states = np.array(states)
            actions = np.array(actions)
            rewards = np.array(rewards)
            next_states = np.array(next_states)
            dones = np.array(dones)
        return states, actions, rewards, next_states, dones

    # Standard Q learning update
    def _q_learning_update(self, state, action, reward, next_state, done):
        # select action and evaluate it using the target network
        target = self.target_net(state)
        for idx in range(len(done)):
            Q_new = reward[idx]
            if not done[idx]:
                Q_new = reward[idx] + self.gamma * torch.max(self.target_net(next_state[

            target[idx][torch.argmax(action[idx]).item()] = Q_new

        return target

    # Double Q learning update
    def _double_q_learning_update(self, state, action, reward, next_state, done):
        # select action using policy network and evaluate it using the target network
        target = self.policy_net(state)
        for idx in range(len(done)):
            Q_new = reward[idx]
            if not done[idx]:
                Q_new = reward[idx] + self.gamma * torch.max(self.target_net(next_state[

            target[idx][torch.argmax(action[idx]).item()] = Q_new

        return target
```

```
    # method to make the agent learn
    def learn(self, state, action, reward, next_state, done):
        # convert the inputs to tensors
        state = torch.tensor(state, dtype=torch.float)
        next_state = torch.tensor(next_state, dtype=torch.float)
        action = torch.tensor(action, dtype=torch.long)
        reward = torch.tensor(reward, dtype=torch.float)
        # (n, x)

        # add one more dimension if the size of inputs is 1
        if len(state.shape) == 1:
            # (1, x)
            state = torch.unsqueeze(state, 0)
            next_state = torch.unsqueeze(next_state, 0)
            action = torch.unsqueeze(action, 0)
            reward = torch.unsqueeze(reward, 0)
            done = (done,)

        if self.double_dqn:
            target = self._double_q_learning_update(state, action, reward, next_state, d

        else:
            target = self._q_learning_update(state, action, reward, next_state, done)

        # predicted Q values with current state
        predicted = self.policy_net(state)

        self.optimizer.zero_grad()
        loss = self.criterion(target, predicted)
        loss.backward()
        for param in self.policy_net.parameters():
            param.grad.data.clamp_(-1, 1)
        self.optimizer.step()
        self._soft_update_target_q_network_parameters()

        self.number_episodes += 1
        self.number_timesteps += 1

        if self.number_timesteps % self.update_frequency == 0:
            self._synchronize_q_networks()
```

helper.py

```
import matplotlib.pyplot as plt
from IPython import display
from Agent import Agent
import numpy as np
from Game import SnakeGameAI

plt.ion()

def update_progress(progress):
    bar_length = 20
    if isinstance(progress, int):
        progress = float(progress)
    if not isinstance(progress, float):
        progress = 0
```

```
    if progress < 0:
        progress = 0
    if progress >= 1:
        progress = 1

    block = int(round(bar_length * progress))

    display.clear_output(wait = True)
    text = "Progress: [{0}] {1:.1f}%".format( "#" * block + "-" * (bar_length - block),
    print(text)

def plot(scores, mean_scores, title=None, clear_output=True):
    if clear_output:
        display.clear_output(wait=True)
    display.display(plt.gcf())
    plt.clf()
    if not title:
        plt.title('Training...')
    else:
        plt.title(title)
    plt.xlabel('Number of Games')
    plt.ylabel('Score')
    plt.plot(scores)
    plt.plot(mean_scores)
    plt.ylim(ymin=0)
    plt.text(len(scores)-1, scores[-1], '{:.2f}'.format(scores[-1]))
    # plt.text(len(mean_scores)-1, mean_scores[-1], str(mean_scores[-1]))
    plt.text(len(mean_scores)-1, mean_scores[-1], '{:.2f}'.format(mean_scores[-1]))
    plt.show(block=False)
    plt.pause(.1)

def moving_average(a, n=3) :
    ret = np.cumsum(a, dtype=float)
    ret[n:] = ret[n:] - ret[:-n]
    return ret[n - 1:] / n

def ax_plot(ax, scores, mean_scores, window=100, title=None):
    ax.set_xlabel('Number of Games')
    ax.set_ylabel('Score')
    if title:
        ax.set_title(title)

    y = np.concatenate((scores[:, np.newaxis], mean_scores[:, np.newaxis]),
                       axis=1)

    if window:
        roll_mean = moving_average(scores, n=window)
        roll_mean = np.pad(roll_mean,
                           (len(scores) - len(roll_mean), 0),
                           'constant', constant_values=np.nan)
        y = np.concatenate((y, roll_mean[:, np.newaxis]), axis=1)

    ax.plot(y)
    ax.text(len(scores)-1, scores[-1], '{:.2f}'.format(scores[-1]), fontsize=8)
    ax.text(len(mean_scores)-1, mean_scores[-1], '{:.2f}'.format(mean_scores[-1]))
```

```
def training_loop(game_kwargs, #dict of kwargs to construct SnakeGameAI
                  model_name,  # name of model to load
                  load_model=False,
                  save_name=None, # name to save best state of model to
                  episode_save = None, # can save after n episodes
                  get_observation = 'relative_snake',
                  greedy=True,
                  epsilon_kwargs={'epsilon': 0.9, 'epsilon_decay':[0.999, 0.995]},
                  double_dqn=False,
                  dueling_dqn=False,
                  num_episodes=1000,
                  plot_update_at_end=False,
                  random_seed=42
                 ):

    # to make runs with different algorithms consistent and comparible, we always
    # want the rats to appear in the same squares for each game. To achieve this
    # we have to set the random seed to be the same for the start of each episode.
    # random.randint() is also called by agent a variable amount of times in between
    # eating each rat. So we have to reset random seed after a rat is eaten.
    rng = np.random.default_rng(seed=random_seed)
    rat_seeds = rng.integers(0, 10e6, size=1000)

    if save_name is None:
        save_name = model_name.split('.')[0] + '_v2' + model_name.split('.')[1]

    game = SnakeGameAI(**game_kwargs, rat_reset_seeds=rat_seeds)

    plot_scores = []
    plot_mean_scores = []
    total_score = 0
    record = 0
    epsilons = []
    agent = Agent(double_dqn=double_dqn,
                  dueling_dqn=dueling_dqn,
                  game=game, greedy=greedy,
                  **epsilon_kwargs)
    if load_model:
        agent.load_model(model_name)
        print('loaded {}'.format(model_name))

    episode = 0

    while episode < num_episodes:
        state = agent.get_observation()
        action = agent.choose_action(state)
        reward, done, score = agent.game.play_step(action)
        new_state = agent.get_observation()
        # remember
        agent.remember(state, action, reward, new_state, done)

        if done:
            episode += 1
            rat_seeds = rng.integers(0, 10e6, size=1000) # new game for episode i will a
            # start the same
            agent.game.rat_reset_seeds = rat_seeds
```

```
            agent.game.reset()
            agent.update_policy()

            states, actions, rewards, new_states, dones = agent.get_memory_sample()
            agent.learn(states, actions, rewards, new_states, dones)

            plot_scores.append(score)
            total_score += score
            mean_score = total_score / agent.number_episodes
            plot_mean_scores.append(mean_score)
            moving_avg = moving_average(np.array(plot_scores), 100)
            if len(moving_avg) > 0:
                if moving_avg[-1] == moving_avg.max():
                    if greedy:
                        agent.save_model(save_name)

            if episode_save == episode:
                agent.save_model(save_name.split('.')[0] + '_' + str(episode_save)
                                 + '_episodes.' + save_name.split('.')[1])

            if plot_update_at_end and not episode == num_episodes:
                update_progress(episode/num_episodes)
                print('{}; episode: {}'.format(model_name.split('.')[0], episode))
            else:
                plot(plot_scores, plot_mean_scores)

    return agent, np.array(plot_scores), np.array(plot_mean_scores)
```

Appendix E rllib code

I have several notebooks in the repo. They are all the same, but used to run different sets of parameters. I do not show ray.tune here because of problems explained in the text, although it is also simple to implement in theory.

```
!pip install "ray[rllib]"
!pip install "gym[atari]" "gym[accept-rom-license]" atari_py
!pip install psutil
# !pip install "gputil" # seemed to create a bug
# below to change mistake in rllib as installed by pip - really strange its not bringing
# in the correct code as version numbers match the github source code
!sed 's/self.unwrapped.np_random.randint/self.unwrapped.np_random.integers/' -i /opt/con

# import urllib.request
# urllib.request.urlretrieve('http://www.atarimania.com/roms/Roms.rar','Roms.rar')
# !apt-get install unrar
# !unrar x Roms.rar
# !mkdir rars
# !sed 's/contrib/contrib non-free/' -i /etc/apt/sources.list
# !apt-get update
# !mv HC\ ROMS.zip   rars
# !mv ROMS.zip  rars
# !python -m atari_py.import_roms rars

# from gym import envs
# print(envs.registry.all())
```

```
!ray --version

!python --version

## Import modules

!sed 's/self.unwrapped.np_random.randint/self.unwrapped.np_random.integers/' -i /opt/con
import ray
import ray.rllib.agents.dqn as dqn
from ray.tune.logger import pretty_print
# from ray.runtime_env import RuntimeEnv
import pickle
import psutil
from matplotlib import pyplot as plt
from pathlib import Path
import numpy as np

config = dqn.DEFAULT_CONFIG.copy()
# for key, val in config.items(): # just to see what the default config is
#     print('{}: {}'.format(key, val))

config = dqn.DEFAULT_CONFIG.copy()
# print(config) # just to see what the default config is
config['env'] = 'PongDeterministic-v4'
# v4 means repeat_action_probability of 0 (always follow issued action)
# deterministic means frameskip set to 4: I'm following
# "Playing Atari with Deep Reinforcement Learning", Mnih et al on this
config['framework'] = 'torch'
config['double_q'] = True   # default is true
config['dueling'] = True   # default is true
config['lr'] = 0.0001
config['adam_epsilon'] = 0.00015
config['hiddens'] = [512]
config['learning_starts'] = 20000
# config['replay_buffer_config']['capacity'] = 1000000 # config['buffer_size'] has been
config['rollout_fragment_length'] = 4
config['train_batch_size'] = 32
config['exploration_config'] = {'type': 'EpsilonGreedy',
                                'initial_epsilon': 1.0,
                                'final_epsilon': 0.01,
                                'epsilon_timesteps': 200000}
config['prioritized_replay_alpha'] = 0.5
# config['disable_env_checking'] = True # set this in case kernel keeps crashing
config['num_gpus'] = 0.2
config['num_workers'] = 14 # this depends on number of CPUs available
config['timesteps_per_iteration'] = 10000
config['model'] = {'dim': 84,
                   'grayscale': True,
                   'zero_mean': True}
# Only for evaluation runs, render the env.
# config['evaluation_config'] = {"render_env": True}

def auto_garbage_collect(pct=80):
    before = psutil.virtual_memory().percent
    if before >= pct:
        gc.collect()
```

```
        print("garbage collection done")
        print("memory from {:.1%} => {:.1%}"
              .format(before/100,
                      psutil.virtual_memory().percent/100)
             )

PKL_FNAME = 'pong_run_data_full_res.pkl'

trainer = dqn.DQNTrainer(config=config)

if Path('./'+PKL_FNAME).exists():
    with open(PKL_FNAME, 'rb') as f:
        run_data = pickle.load(f)
    trainer.restore(run_data['checkpoint'][-1])
else:
    run_data = {'config': config, 'avg_rewards': [], 'max_rewards': [], 'cur_epsilon': [
                'timesteps_since_restore': [], 'timesteps_total': [], 'checkpoint': []}


for i in range(2001):
    # Perform one iteration of training the policy with PPO
    result = trainer.train()
    # print(pretty_print(result))
    print('episode: {}; Mean reward: {:.2f}; Max reward: {}; epsilon: {}'.format(i,
                                                                                  result['epis
                                                                                  result['epis
                                                                                  trainer.get
                                                                                  trainer.get
                                                                                  result['tim
                                                                                 ))
    run_data['avg_rewards'].append(result['episode_reward_mean'])
    run_data['max_rewards'].append(result['episode_reward_max'])
    run_data['cur_epsilon'].append(trainer.get_policy().get_exploration_state()['cur_eps
    run_data['last_timestep'].append(trainer.get_policy().get_exploration_state()['last_t
    run_data['timesteps_since_restore'].append(result['timesteps_since_restore'])
    run_data['timesteps_total'].append(result['timesteps_total'])
    # auto_garbage_collect()
    run_data['checkpoint'].append(None)

    if i % 10 == 0 :
        run_data['checkpoint'][-1] = trainer.save()
        print("checkpoint saved at", run_data['checkpoint'][-1])
        print("memory useage is {:.1%}".format(psutil.virtual_memory().percent/ 100))
        with open(PKL_FNAME, 'wb') as f:
            pickle.dump(run_data, f)

    if i % 100 == 0 and i>0:
        print(pretty_print(result))
```

Also some code to visualise results. Its useful to load save pkl files as one runs training because then results can be manipulated for insights whilst training is still ongoing.

From progress_checker.ipynb

```
from pathlib import Path
from matplotlib import pyplot as plt
import numpy as np
import pickle
```

```
# this model uses tuned parameters from ray project github
PKL_FNAME = 'pong_run_data1.pkl'

if Path('./'+PKL_FNAME).exists():
    with open(PKL_FNAME, 'rb') as f:
        run_data = pickle.load(f)

data = np.concatenate((np.array(run_data['avg_rewards'])[:,np.newaxis],
                       np.array(run_data['max_rewards'])[:,np.newaxis]), axis=1)
y = np.array(run_data['timesteps_total'])
plt.scatter(y, data[:,0].flatten(), alpha=0.5)
plt.scatter(y, data[:,1].flatten(), alpha=0.5);

plt.xlabel('time steps')
plt.ylabel('R')
plt.title('Average and Max reward per episode - "optimum" config', fontsize=12, pad=20)
plt.show()

# this model uses tuned parameters from ray project github
# resulution of screen is 84x84
PKL_FNAME = 'pong_run_data_full_res.pkl'

if Path('./'+PKL_FNAME).exists():
    with open(PKL_FNAME, 'rb') as f:
        run_data = pickle.load(f)

data = np.concatenate((np.array(run_data['avg_rewards'])[:,np.newaxis],
                       np.array(run_data['max_rewards'])[:,np.newaxis]), axis=1)
y = np.array(run_data['timesteps_total'])
plt.scatter(y, data[:,0].flatten(), alpha=0.5)
plt.scatter(y, data[:,1].flatten(), alpha=0.5);

plt.xlabel('time steps')
plt.ylabel('R')
plt.title('Average and Max reward per episode -\nfull instead of reduced res images', fo
plt.show()

# this model uses tuned parameters from ray project github
# lr is doubled from 0.001 to 0.002
PKL_FNAME = 'pong_run_data_lr_high.pkl'

if Path('./'+PKL_FNAME).exists():
    with open(PKL_FNAME, 'rb') as f:
        run_data = pickle.load(f)

data = np.concatenate((np.array(run_data['avg_rewards'])[:,np.newaxis],
                       np.array(run_data['max_rewards'])[:,np.newaxis]), axis=1)
y = np.array(run_data['timesteps_total'])
plt.scatter(y, data[:,0].flatten(), alpha=0.5)
plt.scatter(y, data[:,1].flatten(), alpha=0.5);

plt.xlabel('time steps')
plt.ylabel('R')
plt.title('Average and Max reward per episode -\nlr doubled to 0.0002', fontsize=12, pad
plt.show()
```

Appendix F PPO Code

```
!pip install roboschool gym

!pip install box2d-py

!pip install pybullet

!pip install pygame


################################### Import libraries ###################################


import os
import glob
import time
from datetime import datetime

import torch
import torch.nn as nn
from torch.distributions import MultivariateNormal
from torch.distributions import Categorical

import numpy as np

import gym
# import roboschool
# import pybullet_envs


######################################### set device ##########################################

print("============================================================================================")


# set device to cpu or cuda
device = torch.device('cpu')

if(torch.cuda.is_available()):
    device = torch.device('cuda:0')
    torch.cuda.empty_cache()
    print("Device set to : " + str(torch.cuda.get_device_name(device)))
else:
    print("Device set to : cpu")

print("============================================================================================")




######################################### PPO Policy ##########################################


class RolloutBuffer:
    def __init__(self):
```

```
        self.actions = []
        self.states = []
        self.logprobs = []
        self.rewards = []
        self.is_terminals = []


    def clear(self):
        del self.actions[:]
        del self.states[:]
        del self.logprobs[:]
        del self.rewards[:]
        del self.is_terminals[:]


class ActorCritic(nn.Module):
    def __init__(self, state_dim, action_dim,
                 action_std_init):
        super(ActorCritic, self).__init__()

        self.actor = nn.Sequential(
                        nn.Linear(state_dim, 64),
                        nn.Tanh(),
                        nn.Linear(64, 64),
                        nn.Tanh(),
                        nn.Linear(64, action_dim),
                        nn.Softmax(dim=-1)
                    )


        # critic
        self.critic = nn.Sequential(
                        nn.Linear(state_dim, 64),
                        nn.Tanh(),
                        nn.Linear(64, 64),
                        nn.Tanh(),
                        nn.Linear(64, 1)
                    )

    def set_action_std(self, new_action_std):

        print("--------------------------------------------------------------------------------------------")
        print("WARNING : Calling ActorCritic::set_action_std() on discrete action space
        print("--------------------------------------------------------------------------------------------")


    # maybe get rid of this?
    def forward(self):
        raise NotImplementedError


    def act(self, state):

        # see https://pytorch.org/docs/stable/distributions.html
        #
        action_probs = self.actor(state) # get probabilities given the policy
```

```
        dist = Categorical(action_probs)

        action = dist.sample()
        action_logprob = dist.log_prob(action)

        return action.detach(), action_logprob.detach()


    def evaluate(self, state, action):

        # else:
        action_probs = self.actor(state)
        dist = Categorical(action_probs)

        action_logprobs = dist.log_prob(action)
        dist_entropy = dist.entropy()  # I don't fully understand entropy at the moment
        state_values = self.critic(state)

        return action_logprobs, state_values, dist_entropy


class PPO:
    def __init__(self, state_dim, action_dim, lr_actor, lr_critic, gamma, K_epochs, eps_
                 action_std_init=0.6):

        self.gamma = gamma
        self.eps_clip = eps_clip
        self.K_epochs = K_epochs

        self.buffer = RolloutBuffer()

        # instantiate the NN policy class
        self.policy = ActorCritic(state_dim, action_dim,
                                  # has_continuous_action_space,
                                  action_std_init).to(device)
        self.optimizer = torch.optim.Adam([
                        {'params': self.policy.actor.parameters(), 'lr': lr_actor},
                        {'params': self.policy.critic.parameters(), 'lr': lr_critic}
                    ])

        self.policy_old = ActorCritic(state_dim, action_dim,
                                      # has_continuous_action_space,
                                      action_std_init).to(device)
        # neat way of loading the old policy
        self.policy_old.load_state_dict(self.policy.state_dict())

        self.MseLoss = nn.MSELoss()

    # should be able to get rid of this entirely
    def set_action_std(self, new_action_std):

        print("--------------------------------------------------------------------------------------------")
        print("WARNING : Calling PPO::set_action_std() on discrete action space policy")
        print("--------------------------------------------------------------------------------------------")
```

```
    def decay_action_std(self, action_std_decay_rate, min_action_std):
        print("--------------------------------------------------------------------------------------------")

        print("WARNING : Calling PPO::decay_action_std() on discrete action space policy")

        print("--------------------------------------------------------------------------------------------")


    def select_action(self, state):


        # else:
        with torch.no_grad():
            state = torch.FloatTensor(state).to(device)
            action, action_logprob = self.policy_old.act(state)

        self.buffer.states.append(state)
        self.buffer.actions.append(action)
        self.buffer.logprobs.append(action_logprob)

        return action.item()


    def update(self):

        # Monte Carlo estimate of returns
        rewards = []
        discounted_reward = 0
        for reward, is_terminal in zip(reversed(self.buffer.rewards), reversed(self.buff
            if is_terminal:
                discounted_reward = 0
            discounted_reward = reward + (self.gamma * discounted_reward)
            rewards.insert(0, discounted_reward)

        # Normalizing the rewards
        rewards = torch.tensor(rewards, dtype=torch.float32).to(device)
        rewards = (rewards - rewards.mean()) / (rewards.std() + 1e-7)

        # convert list to tensor
        old_states = torch.squeeze(torch.stack(self.buffer.states, dim=0)).detach().to(d
        old_actions = torch.squeeze(torch.stack(self.buffer.actions, dim=0)).detach().to
        old_logprobs = torch.squeeze(torch.stack(self.buffer.logprobs, dim=0)).detach().


        # Optimize policy for K epochs
        for _ in range(self.K_epochs):

            # Evaluating old actions and values
            logprobs, state_values, dist_entropy = self.policy.evaluate(old_states, old_

            # match state_values tensor dimensions with rewards tensor
            state_values = torch.squeeze(state_values)

            # Finding the ratio (pi_theta / pi_theta__old)
            ratios = torch.exp(logprobs - old_logprobs.detach())
```

```
            # Finding Surrogate Loss
            advantages = rewards - state_values.detach()
            surr1 = ratios * advantages
            ### This is what PPO is all about!! :b
            surr2 = torch.clamp(ratios, 1-self.eps_clip, 1+self.eps_clip) * advantages

            # final loss of clipped objective PPO
            loss = -torch.min(surr1, surr2) + 0.5*self.MseLoss(state_values, rewards) -

            # take gradient step
            self.optimizer.zero_grad()
            loss.mean().backward()
            self.optimizer.step()

        # Copy new weights into old policy
        self.policy_old.load_state_dict(self.policy.state_dict())

        # clear buffer
        self.buffer.clear()


    def save(self, checkpoint_path):
        torch.save(self.policy_old.state_dict(), checkpoint_path)


    def load(self, checkpoint_path):
        self.policy_old.load_state_dict(torch.load(checkpoint_path, map_location=lambda
        self.policy.load_state_dict(torch.load(checkpoint_path, map_location=lambda stora
```

```
print("============================================================================================
```

```
################################### Training ###################################
```

```
####### initialize environment hyperparameters ######

env_name = "CartPole-v1"
# has_continuous_action_space = False

max_ep_len = 400                    # max timesteps in one episode
max_training_timesteps = int(3 * 1e5)   # break training loop if timeteps > max_training

print_freq = max_ep_len * 4     # print avg reward in the interval (in num timesteps)
log_freq = max_ep_len * 2       # log avg reward in the interval (in num timesteps)
save_model_freq = int(2e4)      # save model frequency (in num timesteps)

action_std = None
```

```
################################################################

## Note : print/log frequencies should be > than max_ep_len

################## PPO hyperparameters ##################

update_timestep = max_ep_len * 4      # update policy every n timesteps
K_epochs = 40               # update policy for K epochs
eps_clip = 0.2              # clip parameter for PPO
gamma = 0.99                # discount factor

lr_actor = 0.0003       # learning rate for actor network
lr_critic = 0.001       # learning rate for critic network

random_seed = 42         # set random seed if required (0 = no random seed)

################################################################


print("training environment name : " + env_name)

env = gym.make(env_name)

# state space dimension
state_dim = env.observation_space.shape[0]

# # action space dimension
# if has_continuous_action_space:
#     action_dim = env.action_space.shape[0]
# else:
action_dim = env.action_space.n



########################## logging ##########################

#### log files for multiple runs are NOT overwritten

log_dir = "PPO_logs"
if not os.path.exists(log_dir):
      os.makedirs(log_dir)

log_dir = log_dir + '/' + env_name + '/'
if not os.path.exists(log_dir):
      os.makedirs(log_dir)


#### get number of log files in log directory
run_num = 0
current_num_files = next(os.walk(log_dir))[2]
run_num = len(current_num_files)
```

```
#### create new log file for each run
log_f_name = log_dir + '/PPO_' + env_name + "_log_" + str(run_num) + ".csv"

print("current logging run number for " + env_name + " : ", run_num)
print("logging at : " + log_f_name)

#############################################################


##################### checkpointing ###################

run_num_pretrained = 0      #### change this to prevent overwriting weights in same env_n

directory = "PPO_preTrained"
if not os.path.exists(directory):
      os.makedirs(directory)

directory = directory + '/' + env_name + '/'
if not os.path.exists(directory):
      os.makedirs(directory)


checkpoint_path = directory + "PPO_{}_{}_{}.pth".format(env_name, random_seed, run_num_p
print("save checkpoint path : " + checkpoint_path)

#############################################################


############### print all hyperparameters ###############

print("--------------------------------------------------------------------------------------

print("max training timesteps : ", max_training_timesteps)
print("max timesteps per episode : ", max_ep_len)

print("model saving frequency : " + str(save_model_freq) + " timesteps")
print("log frequency : " + str(log_freq) + " timesteps")
print("printing average reward over episodes in last : " + str(print_freq) + " timesteps

print("--------------------------------------------------------------------------------------

print("state space dimension : ", state_dim)
print("action space dimension : ", action_dim)

print("--------------------------------------------------------------------------------------
print("Initializing a discrete action space policy")

print("--------------------------------------------------------------------------------------

print("PPO update frequency : " + str(update_timestep) + " timesteps")
print("PPO K epochs : ", K_epochs)
print("PPO epsilon clip : ", eps_clip)
print("discount factor (gamma) : ", gamma)

print("--------------------------------------------------------------------------------------
```

```
print("optimizer learning rate actor : ", lr_actor)
print("optimizer learning rate critic : ", lr_critic)

if random_seed:
    print("--------------------------------------------------------------------------------------------")
    print("setting random seed to ", random_seed)
    torch.manual_seed(random_seed)
    env.seed(random_seed)
    np.random.seed(random_seed)

#####################################################################

print("============================================================================================")

################### training procedure ###################

# initialize a PPO agent
ppo_agent = PPO(state_dim, action_dim, lr_actor, lr_critic, gamma, K_epochs, eps_clip,
                # has_continuous_action_space,
                action_std)


# track total training time
start_time = datetime.now().replace(microsecond=0)
print("Started training at (GMT) : ", start_time)

print("============================================================================================")


# logging file
log_f = open(log_f_name,"w+")
log_f.write('episode,timestep,reward\n')


# printing and logging variables
print_running_reward = 0
print_running_episodes = 0

log_running_reward = 0
log_running_episodes = 0

time_step = 0
i_episode = 0


# training loop
while time_step <= max_training_timesteps:

    state = env.reset()
    current_ep_reward = 0

    for t in range(1, max_ep_len+1):

        # select action with policy
        action = ppo_agent.select_action(state)
```

```
        state, reward, done, _ = env.step(action)

        # saving reward and is_terminals
        ppo_agent.buffer.rewards.append(reward)
        ppo_agent.buffer.is_terminals.append(done)

        time_step +=1
        current_ep_reward += reward

        # update PPO agent
        if time_step % update_timestep == 0:
            ppo_agent.update()

        # # if continuous action space; then decay action std of ouput action distributio
        # if has_continuous_action_space and time_step % action_std_decay_freq == 0:
        #     ppo_agent.decay_action_std(action_std_decay_rate, min_action_std)

        # log in logging file
        if time_step % log_freq == 0:

            # log average reward till last episode
            log_avg_reward = log_running_reward / log_running_episodes
            log_avg_reward = round(log_avg_reward, 4)

            log_f.write('{},{},{}\n'.format(i_episode, time_step, log_avg_reward))
            log_f.flush()

            log_running_reward = 0
            log_running_episodes = 0

        # printing average reward
        if time_step % print_freq == 0:

            # print average reward till last episode
            print_avg_reward = print_running_reward / print_running_episodes
            print_avg_reward = round(print_avg_reward, 2)

            print("Episode : {} \t\t Timestep : {} \t\t Average Reward : {}".format(i_epi

            print_running_reward = 0
            print_running_episodes = 0

        # save model weights
        if time_step % save_model_freq == 0:
            print("--------------------------------------------------------------------------
            print("saving model at : " + checkpoint_path)
            ppo_agent.save(checkpoint_path)
            print("model saved")
            print("Elapsed Time  : ", datetime.now().replace(microsecond=0) - start_time
            print("--------------------------------------------------------------------------

        # break; if the episode is over
        if done:
            break

    print_running_reward += current_ep_reward
```

```
        print_running_episodes += 1

        log_running_reward += current_ep_reward
        log_running_episodes += 1

        i_episode += 1


log_f.close()
env.close()



# print total training time
print("============================================================================================")
end_time = datetime.now().replace(microsecond=0)
print("Started training at (GMT) : ", start_time)
print("Finished training at (GMT) : ", end_time)
print("Total training time  : ", end_time - start_time)
print("============================================================================================")
```

```
import pandas as pd
log_num = 0
logs = pd.read_csv('PPO_logs/CartPole-v1/PPO_CartPole-v1_log_{}.csv'.format(log_num))

from matplotlib import pyplot as plt
plt.scatter(x=logs.timestep, y=logs.reward, alpha=0.5)
plt.title('average reward vs timestep')
plt.xlabel('time steps')
plt.ylabel('reward')
```